STAC-R: Subspace-Tracked Activation Compression with Residuals *

Manar Hamed **Yassine Yaakoubi**
Department of Mechanical, Industrial and Aerospace Engineering
Concordia University, Montréal, QC, Canada
manar.hamed@mail.concordia.ca yassine.yaakoubi@concordia.ca

Abstract

Training-memory reduction requires identifying which saved tensors drive peak usage and ensuring high-fidelity backward contractions from compressed states. We formalize saved-activation compression as a *target-surface prediction problem*: estimate, before training, the resident saved surface a compressor can replace and the parameter-gradient error it induces. We introduce STAC-R (Subspace-Tracked Activation Compression with Residuals), a saved-tensor wrapper that stores activations as persistent low-rank coefficients plus a sparse quantized residual and contracts weight gradients directly from these factors, bypassing full decompression. The Saved-Surface Predictor ($\mathrm{SSP} = A_C^+/P$) estimates the peak-memory reduction available on that surface, and a *gradient bridge* links reconstruction error to parameter-gradient cosine similarity. On GPT-2 Medium/Large and Pythia-1.4B, full STAC-R reduces peak memory by 7.2-11.8% with mean gradient cosine ≥ 0.98 in the GPT-2 Medium replay and the Pythia-1.4B 10k-step cohort (5 seeds: $+1.14\%$ validation perplexity); on BERT fine-tuning (SST-2), it reduces peak memory by 12.45% with a 0.15 pp accuracy decrease. Same-surface replays give 0.9813 mean gradient cosine versus 0.9236 for iso-byte SVD and 0.6949 for an untracked subspace. The speed-prioritized Turbo-fast variant reduces GPT-2 Medium peak memory by 9.50% at $0.99\times$ baseline wall time ($+1.82\%$ perplexity at 400 steps), and, at sequence length 4096, the peak memory of Qwen-2.5-7B, Llama-3.1-8B, and Mistral-7B by 6.65-7.46% at 0.90-$0.92\times$ (memory and runtime only). Applied to the blocks that activation checkpointing does not recompute, it raises the GPT-2 Medium saving from 33.84% to 38.58% at similar wall time ($+0.61\%$ versus $+0.02\%$ perplexity); its per-GPU saving stays near 8.9% under DDP up to eight GPUs and rises with FSDP2 world size. Once recomputation has removed the target surface, compression has nothing to act on ($\mathrm{SSP} \to 0$): the memory increases observed on Qwen-2.5-1.5B and Llama-3.2-1B occur against checkpointed baselines, and in a controlled sweep against uncheckpointed baselines both models reduce peak memory by 7-8%.

1 Introduction

Large-transformer training is limited by several distinct memory surfaces: parameters, optimizer state, saved activations, recomputation schedules, allocator fragmentation, and model-specific workspaces. Checkpointing and selective recomputation reduce saved activations by spending additional compute [4, 15]; activation compression replaces tensors retained for backpropagation with a lower-byte representation. The central question is selectivity: peak memory falls when the compressor replaces tensors on the surface that binds the peak and the replacement preserves the backward contraction that determines the weight update.

*Code is available at: https://github.com/Manar-Hamed/STAC-R

40th Conference on Neural Information Processing Systems (NeurIPS 2026).

We treat this as a prediction problem: a saved-activation compressor should specify, before training, whether it can reduce allocator peak for the target model and shape and whether the compressed representation preserves the induced gradient. STAC-R provides this prediction through a target-surface byte ledger and a gradient-fidelity bridge; it wraps selected linear-layer activations, stores each as a persistent tracked low-rank subspace plus a sparse quantized residual, and contracts weight gradients directly from those factors. The memory prediction is capped by the wrapped saved surface; the fidelity prediction bounds update drift by upstream-gradient norm times reconstruction error, together classifying memory-reducing and memory-increasing operating regimes.

Many low-rank methods use similar mathematical objects while targeting different training surfaces: per-batch activation factorizations recompute a batch-local basis every step; optimizer-side methods such as GaLore and SubTrack++ reduce gradient or optimizer-state memory; randomized-subspace optimization (RSO) changes the optimization problem; and checkpointing removes saved activations through recomputation. STAC-R leaves the forward map and the optimizer unchanged, approximates the weight gradients of the wrapped layers, and compresses the saved-activation surface using an amortized basis plus an explicit correction channel.

STAC-R complements activation checkpointing rather than replacing it. In our matched GPT-2 Medium frontier, full recomputation reaches the lowest peak (-67.70% at $1.29\times$ wall time, Table 2); where it is compatible with the training stack and its runtime is affordable, it should be used first. Compression covers the cases this rule leaves open: recomputation latency is the binding constraint, recomputation is unavailable, or recomputation is partial and compression acts on the blocks whose activations remain resident. Full STAC-R is the mechanism reference on which the bounds, telemetry, and residual ablations are defined; the speed-prioritized Turbo-fast variant and its composition with partial recomputation are the deployment points.

Contributions. **Mechanism.** STAC-R is a correctable saved-activation compressor: the tracked basis carries the dominant activation structure, and the sparse residual supplies an explicit correction channel that tightens the gradient-error bound. PhaseProfiler places the GPT-2 Medium r8 peak in the backward path (22.06 GiB vs. 21.33 GiB forward); residual correction contributes 0.14 GiB cumulative delta across 11,520 layer calls.

Prediction. Theorem 1 bounds the replaceable saved-surface bytes from which SSP is formed, and SSP's definition depends only on stored bytes, not on how the payload is formed. Lemma 1 gives the gradient-cosine bridge; Appendix Table 28 maps derived consequences for multi-layer aggregation, rank selection, scheduler-invariant fidelity, activation/optimizer composition, and layer-kind allocation.

Evidence. On GPT-2 Medium/Large and Pythia-1.4B, STAC-R reduces allocator peak by 7.2-11.8% with validation PPL inside the matched-cohort tolerance and mean gradient cosine ≥ 0.98 in the GPT-2 Medium replay and the Pythia 10k cohort; on BERT/SST-2, the 3-seed mean accuracy decrease is 0.15 pp. The comparator package covers same-surface mechanisms (exact activations, STAC-R, batch-local SVD, untracked subspace), per-batch factorizations (SVD/TT/Tucker), residual ablations, Turbo-fast, checkpointing, optimizer-side composition (GaLore, SubTrack++), external comparators (INSTANT, CompAct, ActNN, GACT, OASIS, VeLoRA, COAT), and local reimplementations of the OASIS and PRAC bases.

Deployment and scale. Turbo-fast reduces peak memory at or below baseline wall time on GPT-2 Medium and on 7B/8B decoders at sequence length 4096, adds to partial recomputation when applied to the blocks that are not recomputed, and retains a per-GPU saving under DDP and FSDP2 (Sections 4.2-4.3).

Operating region. The saving requires resident saved activations. The memory-increasing Qwen-2.5-1.5B and Llama-3.2-1B rows of Table 1 were measured against checkpointed baselines, and both models reduce memory against uncheckpointed baselines (Section 4.1).

2 STAC-R Method

STAC-R is implemented as a saved-tensor wrapper around selected modules; the reported decoder cohorts wrap linear layers. For a saved activation matrix $A \in \mathrm{R}^{B\times N}$, it stores

$$\widehat{A} = CQ^{\top} + R_s, \tag{1}$$

where $Q \in \mathrm{R}^{N \times r}$ is an orthonormal Oja-tracked basis, $C = AQ$ stores low-rank coefficients, and R_s is a sparse quantized residual correction. During backward, for upstream gradient $G \in \mathrm{R}^{B \times d_{\text{out}}}$, the weight-gradient contraction is computed as

$$\widehat{\nabla W} = G^\top \widehat{A} = (G^\top C)Q^\top + G^\top R_s. \tag{2}$$

This factor-native contraction distinguishes STAC-R from low-rank storage alone: the residual channel explicitly corrects out-of-subspace error, and the gradient path can be measured directly through cosine telemetry. In fixed-parameter, RNG-pinned backward-pass comparisons at ranks 8, 32, and 128, the minimum gradient cosine over unwrapped parameters was 1.000000000 on GPT-2 Medium and Llama-3.2-1B and at least 0.999999996 on Qwen-2.5-1.5B and Qwen3-1.7B, with a zero mean forward-loss gap. These measurements concern one backward pass at fixed parameters, not subsequent training trajectories.

Tracked basis. The training basis Q is initialized orthonormally, updated online by an Oja streaming subspace rule on the saved activations, re-orthogonalized, and optionally frozen after warmup; each coefficient $C = AQ$ is computed against a basis tracking the dominant activation directions across batches. Exact SVD/PCA computations appear in offline telemetry for the tracked-vs-untracked envelope and the batch-local SVD comparison in Appendix Figure 2. Per-batch SVD incurs a decomposition cost every step; STAC-R amortizes the basis and keeps the backward contraction factor-native (Appendix A.4 gives the formal storage ceiling for batch-local matrix-factor methods).

The residual channel complements the persistent low-rank basis. Sparse-only controls in Appendix A underperform low-rank-only and full STAC-R on both SST-2/BERT and WikiText-103/GPT-2 Large, while residual-on/off replays show a smaller but measurable correction effect. The resulting design is hybrid: amortize the dominant activation structure across batches with Q, then allocate sparse residual entries to tail directions that affect the backward contraction.

Turbo-fast mode. Turbo-fast keeps the same wrapped surface and reduces the per-step basis-refresh and residual-correction overhead when calibration confirms that the surface remains memory-reducing and gradient-faithful. In the cohorts of Sections 4.2-4.3, it runs full STAC-R (rank 8, 0.5% 8-bit residual) for two optimizer steps, then freezes the basis and drops the residual; all model parameters keep training. It is the speed-prioritized variant; quality is reported separately. On Pythia-1.4B, Turbo-fast measures 6.93% peak reduction versus 7.22% for full STAC-R and a 10k PPL increase of 1.22% versus 1.14% for full STAC-R, with lower wall time. Sections 4.2-4.3 evaluate it as the deployment variant.

Backward-binding peak. The measured peak reduction is tied to the backward path. The full PhaseProfiler trace on the GPT-2 Medium STAC-R r8 configuration shows a global peak of 22.06 GiB during backward versus 21.33 GiB during forward, while the backward residual-correction subphase contributes a 0.14 GiB cumulative delta across the $11{,}520$ layer calls in the trace. The wrapped-surface bytes that STAC-R replaces are therefore largest at the upstream-gradient contraction, the step controlled by Lemma 1.

3 Target-Surface Prediction

Two conditions determine whether STAC-R reduces peak memory for a given configuration: the wrapped saved surface must be large enough to change allocator peak, and the compressed state must preserve the backward contraction that forms the weight gradient. The target-surface ceiling addresses memory; the gradient bridge addresses fidelity. Appendix Table 28 collects the derived consequences used in the evaluation and points to their proofs and calibration evidence.

Definition 1 (Wrapped target surface)**.** *For model M and training configuration* cfg*, let $W(M, \mathrm{cfg})$ be the saved-activation bytes intercepted during one training step. Let $S_C(M, \mathrm{cfg})$ denote the compressed payload replacing that same surface. Persistent bases, temporary workspaces, allocator fragmentation, and other terms are tracked separately by allocator/NVML measurement.*

Theorem 1 (Target-surface memory ceiling)**.** *For the activation-side byte ledger of any saved-activation compressor whose replacement payload is restricted to the wrapped target set,*

$$\begin{aligned} A_C(M, \mathrm{cfg}) &= W(M, \mathrm{cfg}) - S_C(M, \mathrm{cfg}), \\ A_C^+(M, \mathrm{cfg}) &= \max\{A_C(M, \mathrm{cfg}), 0\} \le W(M, \mathrm{cfg}). \end{aligned} \tag{3}$$

Larger total activation volume yields larger memory savings when the added tensors lie inside the wrapped target surface and the replacement payload is smaller than the original saved state.

Remark 1. *The inequality bounds the saving inside the saved-payload ledger; it is not by itself a bound on the change in total CUDA allocator peak, which also depends on allocator fragmentation, optimizer state, parameters, persistent bases, temporary workspaces, and logging tensors. Bases and workspaces are excluded from S_C and are measured separately by allocator/NVML audits. Configurations with small W/P, such as small activation-fraction fine-tuning, have a limited peak-memory ceiling even under a high compression ratio, and when recomputation has already discarded the wrapped activations, $A_C^+ \to 0$, as in the checkpointed Qwen and Llama configurations (Section 4.1).*

Definition 2 (Saved-Surface Predictor)**.** *For a fixed method and configuration, define*

$$\mathrm{SSP}(M, \mathrm{cfg}) = \frac{A_C^+(M, \mathrm{cfg})}{P(M, \mathrm{cfg})}, \tag{4}$$

where P is the baseline allocator peak. SSP is a pre-run screening statistic: small wrapped surfaces and large replacement payloads make it small even when gradient cosine is high, while persistent bases and temporary workspaces lie outside the ledger and can remove a positive predicted saving. SSP is therefore paired with measured or predicted gradient fidelity, allocator measurement, and system constraints such as checkpointing compatibility. Its definition uses only W, S_C, and P, so it can be computed for any compressor that replaces saved tensors with a smaller payload (Appendix Table 24 compares five payloads).

Lemma 1 (Activation-error to gradient-error bridge)**.** *Let $A, \widehat{A} \in \mathrm{R}^{B\times N}$, let $E = A - \widehat{A}$, and let $G \in \mathrm{R}^{B\times d_{\mathrm{out}}}$ be the upstream gradient at the wrapped linear layer. Then*

$$\Delta\nabla W = G^\top A - G^\top \widehat{A} = G^\top E, \qquad \|\Delta\nabla W\|_F \le \|G\|_2 \, \|E\|_F \le \|G\|_F \, \|E\|_F . \tag{5}$$

If $\nabla W \neq 0$ and $\|\Delta\nabla W\|_F \le \epsilon_G \|\nabla W\|_F$ with $\epsilon_G < 1$, then $\widehat{\nabla W} \neq 0$ and the sharper gradient-space bound is

$$\cos(\nabla W, \widehat{\nabla W}) \ge \sqrt{1 - \epsilon_G^2}. \tag{6}$$

Appendix B.1 also gives the activation-space relative-error bridge $\epsilon_G \le \kappa_G \|E\|_F / \|A\|_F$ with $\kappa_G = \|G\|_2 \|A\|_F / \left\|G^\top A\right\|_F$. Appendix Table 29 verifies the bound on 108 probed layers across BERT-base, GPT-2 124M, and GPT-2 Large; the residual correction reduces the measured gradient-error proxy by $3.0\times$-$50.6\times$ across the three architectures.

4 Evaluation

Evaluation uses matched, within-protocol comparisons. Table 1 and Appendix Table 5 report training cohorts whose allocator peak, quality, and runtime are compared within the same model, data, schedule, and batch/sequence setting; Tables 2 and 3 add a matched recomputation frontier and Turbo-fast cohorts across model families, sizes, sequence lengths, and hardware, each row compared only with the baseline of its own cohort. Appendix Figure 1 is the quantitative target-surface prediction check, pairing the memory ceiling with 240-step same-shape gradient-fidelity replay.

The evidence is organized around three evaluation questions. First, the prediction/fidelity subsection pairs the target-surface memory ceiling with gradient-cosine telemetry. Second, the mechanism subsection compares the same wrapped surface under exact activations, STAC-R, batch-local SVD, and an untracked subspace control; adjacent appendix tables add per-batch SVD/TT/Tucker runtime measurements, sparse-only controls, residual-on/off sweeps, and reimplemented OASIS and PRAC bases. Third, the systems sections report Turbo-fast, its composition with checkpointing, scaling in model size, sequence length, and GPU count (Sections 4.2-4.3), and GaLore, SubTrack++, gradient accumulation, and external comparator positioning. Main-text tables report matched comparisons only; the cross-benchmark positioning grid, whose INSTANT rows use a different code path, is Appendix Table 6, and Appendix Table 9 gives the full evidence map.

4.1 Memory, Quality, and Time

Table 1 groups the quality, memory, and runtime deltas of the displayed STAC-R configurations. Across the memory-reducing summary cohorts, matched long-horizon validation-PPL increase

Table 1: Quality, memory, and runtime summary. Memory is allocator peak relative to a matched baseline; quality is validation PPL except for SST-2 accuracy. Pythia uses the allocator-audited 2k memory row and the matched 10k 5-seed PPL/cosine cohort; Appendix Table 5 reports unmixed allocator rows. The Qwen and Llama rows are measured against a gradient-checkpointed baseline (Section 4.1). Single RTX 5090 under the protocol of Section 5; seed 42 unless noted.

Model	Surface	Mem. Δ%	Quality Δ	Time Δ×	Cos. (mean)	Notes
GPT-2 Medium	MLP-linear (rank 8)	-9.20	+1.46% PPL	1.95x	0.9813	500-step cohort
GPT-2 Large	MLP-linear (rank 8)	-11.75	+1.15% PPL	2.40x	n/a	MLP-only
Pythia-1.4B	MLP-linear (rank 8)	-7.22	+1.14% PPL	1.90x	0.9917	2k memory; 10k PPL
BERT-base SST-2	MLP-linear (rank 8), 3-epoch	-12.45	-0.15 pp acc	2.04x	n/a	3-seed SST-2
Qwen-2.5-1.5B	SwiGLU MLP rank 8, b=2 s=1024	+66.23	+0.92% PPL	1.41x	0.9909	ckpt. baseline; 5k
Llama-3.2-1B	SwiGLU MLP rank 8, b=2 s=1024	+75.99	+8.76% PPL	1.31x	0.9906	ckpt. baseline; 5k

remains within 1.5% on decoder pretraining; the Pythia 5k scheduler-warmup measurement reaches $+2.56\%$, while the 10k matched cohort is the primary Pythia quality cohort. On the BERT/SST-2 fine-tuning cohort, the 3-seed mean accuracy decrease is 0.15 pp with worst seed 0.34 pp. Mean gradient cosine is 0.9813 in the GPT-2 Medium replay and 0.9917 in the Pythia 10k cohort; the short replays of Appendix Table 8 give lower means for Pythia (0.9672) and the GPT-2 Large MLP+attention surface (0.9415).

Table 1 also includes two SwiGLU rows that increase memory while preserving gradient fidelity. Qwen-2.5-1.5B and Llama-3.2-1B retain high 5k gradient fidelity (Qwen mean/min cosine $0.9909/0.9092$, Llama $0.9906/0.9061$), while allocator peak rises by $+66.23\%$ and $+75.99\%$ at b=2 seq=1024; a Llama b=1 seq=4096 shape retest remains memory-increasing at $+38.84\%$ peak with cosine $0.9839/0.9678$. These rows were measured against a gradient-checkpointed baseline, where recomputation has already discarded the saved activations that STAC-R targets. Sweeping only the baseline's recomputation setting (Appendix Table 23), the peak change is -7.08% (Llama) and -7.80% (Qwen) without recomputation, -4.34% and -4.78% with every second block recomputed, and $+0.03\%$ and -0.16% with all blocks recomputed; recomputing every second block and compressing the others gives -22.01% and -22.09%, against -18.48% and -18.14% for that recomputation alone. Both models are therefore memory-reducing when their saved activations are resident, and the original rows do not establish an architectural incompatibility with SwiGLU. The controlled sweep does not reproduce the magnitude of the original increases. The unmixed per-cohort allocator measurements are in Appendix Table 5.

Appendix Table 5 gives the complete memory-quality-time tradeoff for the matched-cohort comparison: allocator peak, validation PPL, wall time, and within-cohort memory reduction for GPT-2 Medium, Pythia-1.4B, GPT-2 Large+GaLore, and the GPT-2 Large surface-expansion cohort. Across the primary GPT-2 Medium/Large cohorts, full STAC-R reports 9.2-11.8% peak-memory reduction, 1.15-1.46% PPL increase, and 1.95-2.40× wall time relative to the matched baselines. The GPT-2 Medium checkpointing-only row reaches 8.328 GiB and 17.3569 PPL at 182.2 s, so recomputation is the lower-memory choice when exact checkpointing is compatible and its runtime tradeoff is acceptable. Pythia-1.4B is the second decoder cohort with memory savings. Turbo-fast is the speed-prioritized variant: it reduces wall time relative to full STAC-R with the PPL deltas reported in Appendix Tables 5 and 11; Sections 4.2-4.3 evaluate it as the deployment variant against recomputation and at scale.

Pythia matched 10k 5-seed cohort. The 10k matched cohort over seeds 42, 123, 456, 789, and 1024 gives baseline 13.172 ± 0.106 PPL and STAC-R 13.322 ± 0.047, for a $+\mathbf{1.14}\%$ mean PPL gap; Turbo-fast is 13.333 ± 0.062 ($+1.22\%$). Extending from 3 to 5 seeds changes the STAC-R mean gap from $+1.10\%$ to $+1.14\%$; both remain within the 1.5% matched-cohort tolerance. The 5k scheduler/fidelity measurement reaches $+2.56\%$ PPL while maintaining high cosine; Appendix A reports it separately from the 10k quality cohort. The 5-seed cosine statistics include the cold step-0 measurement (STAC-R mean/min $0.9917/0.8930$); post-cold medians round to 1.0 at the logged precision, and cross-seed PPL standard deviation contracts from baseline 0.106 to STAC-R 0.047.

Table 2: Recomputation frontier and composition with Turbo-fast on GPT-2 Medium (WikiText-103, RTX 5090, $b = 8$, $s = 1024$, 400 steps, three seeds; bf16 autocast, 8-bit AdamW). All deltas and time ratios are relative to the uncheckpointed baseline of this cohort (23.566 GiB, PPL 15.3439). Turbo-fast is applied to the MLP surface of the blocks that are not recomputed. PPL for the all-recomputed rows is reported as "exact"; with every block recomputed, the Turbo-fast arm has nothing to compress and is byte-identical to recomputation alone.

Recomputed blocks	Turbo-fast	Peak (GiB)	Δ peak (%)	Time (×)	Δ PPL (%)
0	no	23.566	0.00	1.000	0.00
0	yes	21.327	−9.50	0.990	+1.82
3	no	21.570	−8.47	1.037	−0.02
3	yes	19.612	−16.78	1.029	+1.52
6	no	19.583	−16.90	1.073	+0.01
6	yes	17.899	−24.05	1.066	+1.16
12	no	15.591	−33.84	1.144	+0.02
12	yes	14.475	−38.58	1.139	+0.61
24 (all)	no	7.612	−67.70	1.292	exact
24 (all)	yes	7.612	−67.70	1.292	exact

4.2 Turbo-fast and Activation Checkpointing

Table 2 places Turbo-fast on the recomputation frontier in a fresh 400-step GPT-2 Medium cohort; absolute values are compared only within the table. Turbo-fast alone saves 9.50% at $0.990\times$ baseline time with a $+1.82\%$ PPL change. In the measured recomputation sweep, denser schedules generally reduced peak memory at greater wall-time cost. In a separate 300-step cohort, where Turbo-fast saves 9.51% at $0.991\times$, the sparsest recomputation schedules tested cost 1.026-$1.050\times$ (Appendix A.6). When the two are combined, Turbo-fast is applied only to the MLP surfaces of blocks that are not recomputed. At each partial density, the composed arm reaches a lower peak than recomputation alone at similar wall time and a higher PPL cost; with 12 of 24 blocks recomputed, it gives -38.58% versus -33.84% at $1.139\times$ versus $1.144\times$, with $+0.61\%$ versus $+0.02\%$ PPL. The additional reductions are approximately proportional to the number of compressed blocks (2.239, 1.958, 1.684, and 1.116 GiB for 24, 21, 18, and 12 blocks, about 0.093 GiB per block), an empirical pattern in this sweep. With all 24 blocks recomputed, nothing remains to compress and the two arms are byte-identical; full recomputation gives the lowest peak in the table (-67.70% at $1.292\times$).

In a separately reported GPT-2 Medium cohort with the same stated setup ($b = 8$, $s = 1024$, 400 steps, three seeds), full STAC-R saves 9.40% at $1.69\times$ with $+0.99\%$ PPL. With every second block recomputed, applying it to the other blocks gives -38.53% at $1.49\times$ ($+0.44\%$ PPL), whereas applying it to the recomputed blocks gives -33.84% at $1.76\times$, no more than recomputation alone. Compressing the non-recomputed blocks also adds to recomputation on GPT-2 Large (-36.08% versus -32.36%) and on Llama-3.2-1B and Qwen-2.5-1.5B (Section 4.1), and STAC-R removes a further 13.3% of per-rank peak on top of checkpointing under TP$= 2$ (Table 21).

4.3 Scaling Across Models, Sequence Length, and GPUs

Table 3 extends Turbo-fast to 17 further open models, up to 8B parameters, on RTX 5090 and H100 GPUs. Turbo-fast reduces peak memory in every configuration except Qwen-2.5-7B at $s = 1024$, including both SwiGLU models of Table 1. Across these models and GPT-2 Medium, it ran at or below baseline wall time on 16 of 18; the exceptions are GPT-2 XL and one of the nine further models.

Composition at scale. Composition with partial recomputation, with every fourth block recomputed (4-8 blocks, depending on depth) and Turbo-fast applied to the others, was reported separately from the Turbo-fast-only runs of Table 3, so the two are not paired row by row. Llama-3.2-1B, Qwen-2.5-1.5B, and Qwen3-1.7B reach -18.83%, -21.83%, and -19.18% at $0.999\times$, $1.014\times$, and $1.018\times$ baseline time; the three 7B/8B models reach reductions of 16.17-18.15%, each at or faster than baseline time; and the nine further models reach 8.63-24.10% under the protocol of their Table 3 row.

Sequence length. For Qwen-2.5-7B at $b = 1$, Turbo-fast changes peak by $+0.03\%$ at $s = 1024$, where the saved surface is too small for any tested method to reduce peak, and by -7.38% at

Table 3: Turbo-fast across model families and sizes (GPT-2 Medium: Table 2). Each row compares Turbo-fast with its own baseline in the same cohort and machine; Δ is the change in allocator peak and Time the wall-time ratio. RTX 5090 rows use three seeds; H100 rows use bf16 master weights, $b = 1$, and two seeds. Precision is the master-weight dtype, with bf16 autocast and 8-bit AdamW. The table reports memory and runtime only.

Model	Setting	Baseline peak (GiB)	Δ peak (%)	Time ($\times$)
RTX 5090 (32 GiB), $s = 1024$				
GPT-2 Large	fp32, $b = 4$	23.498	-9.10	0.994
GPT-2 XL	bf16, $b = 2$	19.826	-7.47	1.030
Llama-3.2-1B	fp32, $b = 2$	16.225	-13.80	0.939
Qwen-2.5-1.5B	fp32, $b = 2$	21.585	-16.26	0.951
Qwen3-1.7B	fp32, $b = 2$	23.574	-12.93	0.954
9 further models[a]	fp32/bf16, $b = 2$	14.3-25.5	-17.46 to -4.31	0.919-1.036
H100, bf16, $b = 1$				
Qwen-2.5-7B	$s = 1024$	42.774	$+0.03$	0.977
Qwen-2.5-7B	$s = 4096$	64.890	-7.38	0.902
Llama-3.1-8B	$s = 4096$	67.086	-6.65	0.922
Mistral-7B-v0.3	$s = 4096$	59.798	-7.46	0.919

[a] Pythia-1.4B [2], StarCoder2-3B [18], Phi-2 [12], Falcon-RW-1B [20] (GELU); Gemma-2-2B [9], Gemma-3-1B [10] (GeGLU); OPT-1.3B [28] (ReLU); OLMo-2-1B [25], StableLM-2-1.6B [1] (SwiGLU); ranges are over models.

Table 4: Saved-Surface Predictor (SSP) operating rule. SSP is A_C^+/P: positive wrapped-surface byte saving divided by baseline allocator peak.

Row	SSP %	Memory outcome	Quality	Cos.	Decision
GPT-2 Medium STAC-R	8.29	9.20% saved	+1.46% PPL	0.9813	Full or Turbo-fast
GPT-2 Medium Turbo-fast	8.29	**9.27% saved**	**+1.01% PPL**	**0.9938**	Runtime priority
Pythia-1.4B STAC-R	7.21	**7.22% saved**	**+1.14% PPL**	**0.9917**	Full mode
Pythia-1.4B Turbo-fast	7.36	6.93% saved	+1.22% PPL	0.9912	Runtime priority
BERT-base SST-2	–	12.45% saved	-0.15 pp acc	–	Downstream check
Qwen/Llama SwiGLU	–	memory-increasing vs. ckpt. baseline	+0.92/+8.76% PPL	$\approx$0.991	Recompute; compress non-recomputed blocks
GPT-2 checkpoint-compatible	–	**65.66% saved by ckpt.**	matched PPL	–	Recompute

$s = 4096$ (Table 3). On Qwen-2.5-1.5B at $b = 1$ on the RTX 5090, Turbo-fast's time ratio falls from $1.014\times$ at $s = 1024$ to $0.931\times$ and $0.901\times$ at $s = 2048$ and 4096.

Distributed training. Under DDP on GPT-2 Medium ($b = 4$ per rank), Turbo-fast's per-rank saving stays at 8.92-8.95% across world sizes 1-8. Under FSDP2, which shards parameters, gradients, and optimizer state but not each rank's activations, its saving on Qwen-2.5-1.5B ($b = 2$ per rank) grows from 6.51% at world size 1 to 8.34% and 9.99% at world sizes 2 and 4. Both compose with partial recomputation (Appendix A.6); all values are per-rank peaks measured on every rank.

4.4 Prediction Check and Fidelity Telemetry

Decision procedure. SSP is evaluated on the saved tensors that remain resident after recomputation and other graph transformations, and Table 4 turns it into operating choices. (i) Where exact recomputation is compatible and its runtime fits the budget, use it first; it reached the lowest peak in our measurements. (ii) If recomputation has removed the target tensors, $\mathrm{SSP} \to 0$ and compression has nothing to act on. (iii) If the target tensors are resident, SSP estimates the available saving and Lemma 1 bounds the fidelity cost; Turbo-fast is the deployment choice when runtime is binding, with quality checked at the intended horizon. (iv) When both are used, compress only blocks that are not recomputed; in our tests, compressing recomputed blocks changed peak by at most 0.21 percentage points. Optimizer-side methods act on a separate surface and are added where a measured composition supports it. Under mixed precision, tensor parallelism, or fused kernels, SSP and fidelity are evaluated in the intended configuration; a fused kernel that does not expose its input to saved-tensor hooks keeps that tensor outside the wrapped surface. Within the surface, wrap the MLP linears first, raise the rank when quality is binding, and add attention when memory is binding (Appendix Table 26).

The SSP table also explains why high activation-compression ratios translate into single-digit or low-teens total-peak changes. For GPT-2 Medium full STAC-R, SSP predicts an 8.29% allocator reduction and the audited run measures 9.20%; for the Pythia-1.4B shape, the prediction is 7.21% and the measured reduction is 7.22%. SSP is therefore an estimate rather than a bound on the measured change. The relevant quantity is the replaceable saved surface as a fraction of the total allocator peak, not the compression ratio of the wrapped tensors in isolation. Figure 4 in Appendix A calibrates SSP against observed allocator saving and visualizes the operating rule.

4.5 Mechanism and Comparator Evidence

Appendix Figure 2 and Table 7 give the same-surface mechanism study. Exact activations define the reference backward contraction. Batch-local SVD measures the low-rank reconstruction available from a decomposition on the current batch. The untracked subspace control measures whether low-byte storage alone is sufficient on the same wrapped surface. STAC-R measures the proposed mechanism: an amortized Oja-tracked basis, sparse residual correction, and factor-native backward contraction.

The local and full-model measurements separate reconstruction geometry from update fidelity. At rank 8, batch-local SVD and STAC-R have similar local per-layer cosine (0.670 versus 0.666), but the full-model replay favors STAC-R: 0.9813 mean gradient cosine versus 0.9236 for the iso-byte SVD control and 0.6949 for the matched-rank untracked control. The cosine-equivalent proxy $\epsilon_{\cos} = (1 - \cos)/(1 + \cos)$ decreases from 0.180 for the untracked control to 0.0094 for STAC-R, a $19.1\times$ reduction at the same matched-surface stored-byte budget. The seed-43 replay reproduces the untracked-control value with mean cosine 0.6950, and the iso-byte companion shows that the gap persists when the untracked control is allocated $10.2\times$ the stored bytes of STAC-R r=8.

The residual and factorization controls answer different mechanism questions. Residual on/off replays give a smaller but measurable correction-channel effect across ranks: cosine-bound γ_ϵ is 0.193 at r8, 0.175 at r32, 0.149 at r128, and 0.099 at r307. Appendix Table 14 adds the training-time counterpart: in a no-AMP/no-checkpointing SST-2/BERT slice, per-step recomputed SVD/TT/Tucker measure 76-117 tok/s versus 730 tok/s for STAC-R. The runtime gap is the decomposition cost incurred every batch by per-step recomputed factorization, compared with the amortized basis used by STAC-R. The same residual design changes the matched-budget operating point: Appendix Table 16 shows a throughput gain over pure-low-rank storage that grows from $+7.4\%$ on BERT-base to $+55.2\%$ on GPT-2 XL while keeping the quality gap small in the decoder rows. Appendix Figure 7 contains the full rank-sweep mechanism view.

Basis controls and residual cost. Appendix Table 24 compares STAC-R (Oja basis plus residual) at rank 8 on the same MLP surface and with the same factor-native backward against the Oja basis without the residual, local reimplementations of the OASIS and PRAC bases, and an untracked basis. All five reach the same peak within 0.11 percentage points, while STAC-R changes PPL by $+0.92\%$ against $+1.64\%$ to $+2.02\%$ for the others. Beyond the single operating point of Table 19, the residual improves quality at nearly constant peak memory but costs runtime; a single-chunk top-k selector reduces full STAC-R from $1.681\times$ to $1.376\times$ baseline time at the same reported peak and nearly unchanged PPL (Appendix A.6).

4.6 Stored Bytes and Composition

STAC-R also composes with optimizer-side and training-loop methods. The 5k GaLore composition cohort contains three matched seeds: baseline, GaLore, and STAC-R+GaLore reach peaks of 14.614, 10.894, and 9.591 GiB respectively, so full STAC-R reduces peak by 1.302 GiB relative to GaLore-alone (11.96% marginal), raises PPL by $+2.65\%$ over GaLore, and runs at $1.99\times$ GaLore-alone time. In the same 3-seed cohort, Turbo-fast+GaLore matches the full STAC-R+GaLore allocator point within 0.005 GiB (9.587 versus 9.591 GiB), runs at $0.67\times$ the full-stack wall time, and has lower mean PPL (16.5898 ± 0.0092 versus 16.6596 ± 0.0262). The Turbo-fast+GaLore row is the speed-prioritized optimizer composition: 12.00% marginal peak reduction over GaLore-alone, $+2.22\%$ PPL over GaLore, and $1.34\times$ GaLore-alone time. Appendix Table 18 reports the same surface-separation point for SubTrack++: after optimizer-side rank-4 compression, adding STAC-R reduces allocated peak by another 13.1%. The freed bytes also change what fits on a fixed device: GPT-2 XL with 8-bit AdamW trains at $b = 6$ instead of $b = 5$ on a single RTX 5090 (Appendix

Table 20), and under TP$= 2$ with checkpointing the maximum verified batch rises from 22 to 26 (Table 21). The main memory claims remain the unmixed STAC-R saved-activation cohorts.

5 Systems and Reproducibility

The original audited configurations use seed 42 unless otherwise stated, a single RTX 5090 (32 GiB) host, AMP `auto` with the 8-bit AdamW optimizer, `torch.cuda.max_memory_allocated` as the allocator-peak source, and NVML `memory.used` as an external cross-check; multi-seed cohorts report seeds 123/456 and, for the Pythia 10k extension, seeds 789/1024. Reported peaks use allocator/NVML measurements, not the two-step accounting ledger; Appendix A.5 gives PhaseProfiler phase timings. A five-repeat Pythia Turbo-fast replay produces byte-identical allocator peak and reserved memory across runs. Residual quantization is the largest payload contributor: disabling it raises stored bytes from 43.8 MiB to 51.0 MiB ($+16.4\%$); disabling coefficient quantization raises it to 46.8 MiB ($+6.8\%$). Appendix Table 17 reports the 6-configuration Turbo evidence; Table 4 is derived from the SSP calibration data released with the code.

The added training cohorts use three seeds on RTX 5090 and two on H100 where specified in the corresponding experiment protocols. The DDP and FSDP2 cohorts run on an $8\times$RTX 5090 host. RNG-pinned probes are described separately and are not included in this blanket training-cohort seed count. Precision denotes the master-weight dtype, with bf16 autocast and 8-bit AdamW throughout except the FSDP2 cohort (PyTorch AdamW, bf16 states); peak is `torch.cuda.max_memory_allocated` over the complete step, and every relative number uses the baseline of its own cohort and machine. In the separately reported GPT-2 Medium cohort of Section 4.2, per-seed allocator peaks were identical on every reported row (standard deviation 0.000000 GiB across seeds 42/123/456). Relative savings reproduced on an H100 for the frontier arms rerun there (-9.49% versus -9.50% for Turbo-fast, -67.67% versus -67.70% with all blocks recomputed).

6 Related Work

Appendix Table 13 separates related methods by the compressed object, whether the standard training problem is preserved, the distinction from STAC-R, and the evidence type. The main distinction is taxonomic: methods can share low-dimensional structure while acting on different tensor surfaces or different training objectives.

Among saved-activation methods, INSTANT is the closest per-batch activation-factor comparator. Appendix Tables 6 and 15 report an official-code reproduction on SST-2/BERT, GPT-2 Large/WikiText-103, and MRPC/BERT, with deltas measured against each method's own baseline because the released code paths differ. The short-sequence reproduction exposes the same factor-storage pressure as the SVD ceiling: in the $L = 128$ official-code row used for this comparison, the released INSTANT setting reaches $1.0\times$ effective activation compression while STAC-R r8 remains at $88.7\times$ on the same shape. CompAct is included through a bounded local measurement. OASIS and PRAC have no usable public implementation (PRAC releases no code, and the OASIS repository contained no code when checked in July 2026), so Appendix Table 24 reimplements their bases under our matched-surface protocol with the same factor-native backward; this does not reproduce each method's own pipeline. VeLoRA and COAT are cited as related mechanism or whole-stack methods without local matched saved-surface measurements [8, 24, 6, 19, 26]. The same-surface untracked-subspace control follows Li et al. [16]; it isolates the MLP-linear wrapped surface used in the mechanism study and sits inside the broader comparator suite.

ActNN and GACT are reported as original-paper comparators. ActNN reports 2-bit activation-compressed training with $12\times$ activation-memory reduction and $6.6\times$-$14\times$ larger batches on its evaluated computer-vision tasks; GACT reports up to $8.1\times$ activation-memory reduction and $4.2\times$-$24.7\times$ larger batches across convolutional, transformer, and graph models [3, 17]. The compatibility-patched ActNN/GACT probes in our released code are engineering diagnostics; Appendix Table 5 uses local total-CUDA-peak and gradient-cosine comparators with matched measurement protocols.

Checkpointing and selective recomputation trade extra compute for lower activation storage [4, 15], and Dynamic Tensor Rematerialization (DTR) performs tensor eviction and rematerialization as an online runtime strategy that supports dynamic graphs [14]. These methods are lossless and discard saved tensors, whereas STAC-R keeps a lossy compressed state without forward replay;

static recomputation and compression compose when applied to disjoint blocks (Section 4.2). Our frontier uses static selective recomputation as the rematerialization reference; we did not evaluate DTR directly, since its released prototype targets a PyTorch/CUDA stack that predates our hardware. Optimizer methods such as GaLore, APOLLO, and SubTrack++ target gradient or optimizer-state memory, so they are treated as composition axes for saved-activation compression [29, 30, 23]. RSO is related in its use of randomized low-dimensional structure and changes the optimization problem by training through low-rank perturbation subproblems; STAC-R preserves the original forward map and optimizer, approximating the wrapped weight gradients, while compressing the saved-activation path [5]. Evaluation substrates GPT-2, BERT, Pythia, Qwen, Llama, and Mistral are cited as model/dataset context [22, 7, 2, 21, 27, 11, 13].

7 Operating Regimes

STAC-R reduces peak memory when the saved-activation surface that remains resident is large relative to allocator peak and the stored representation remains gradient-faithful; the evaluated recipe covers transformer Linear/Conv1d saved activations under the reported AMP/optimizer settings. Recomputation, and fused kernels that do not expose their inputs to saved-tensor hooks, keep tensors out of the wrapped surface. The SwiGLU layout is not itself a barrier (Section 4.1; Table 3); the memory-increasing rows of Table 1 remain negative results under their original checkpointed protocol. The saved surface can also be too small without checkpointing, as for Qwen-2.5-7B at $b = 1$, $s = 1024$. Arbitrary graph-message memory, small activation-fraction fine-tuning, and attention wrapping with the tested split-QKV remain separate operating regimes (Table 22), and the first adaptive-rank schedule, `c_fc=4,c_proj=12`, violates the per-layer cosine floor and loses to uniform rank.

Quality is a separate limitation. The Pythia-1.4B 10k-step result is specific to that cohort; quality should be validated at the intended training horizon. Quality at 7B/8B scale is not reported, and the distributed results cover per-rank memory, not communication overhead or throughput scaling.

8 Conclusion

STAC-R treats saved-activation compression as a prediction problem with explicit bounds. Theorem 1's saved-surface byte ceiling, the SSP statistic built on it, and Lemma 1's gradient-cosine bridge together explain the memory-reducing regimes (9.2-11.8% peak savings on GPT-2 Medium/Large, -7.22% on Pythia-1.4B, -12.45% on full BERT/SST-2) and the regimes in which recomputation has already removed the target surface. For full STAC-R on the original summary cohorts, validation PPL stays within 1.5% at matched long horizon, or 0.15 pp mean SST-2 accuracy decrease (0.34 pp worst seed), with mean gradient cosine ≥ 0.98 in the GPT-2 Medium replay and the Pythia 10k cohort.

Same-surface replays show that STAC-R preserves the backward contraction more faithfully than iso-byte SVD and untracked-subspace controls, ablations identify the residual as a correction channel, and GaLore, SubTrack++, and checkpointing measurements locate STAC-R relative to optimizer-side and recomputation systems.

For deployment, Turbo-fast reduces peak memory at or below baseline wall time on most tested models, including 7B/8B decoders, and adds to partial recomputation when applied to the blocks that are not recomputed. The saving requires resident saved activations, so it vanishes where recomputation has already removed them. Future work follows directly: a tensor-level policy that chooses among retaining, compressing, and rematerializing each tensor from SSP's byte term, Lemma 1's fidelity cost, and recomputation cost; a constrained per-layer cosine-floor allocator that allocates bytes by $\|G_\ell\|_F \|E_\ell\|_F$ mass; and quality evaluation over longer horizons, at 7B/8B scale, and on text distributions beyond the 500-step OpenWebText transfer check. The contribution is a saved-activation compressor whose memory and gradient-fidelity conditions can be checked before training begins.

Acknowledgments and Disclosure of Funding

This work received no third-party funding. The authors declare no competing interests.

References

[1] Marco Bellagente, Jonathan Tow, et al. Stable LM 2 1.6B technical report, 2024.

[2] Stella Biderman, Hailey Schoelkopf, Quentin Gregory Anthony, Herbie Bradley, Kyle O'Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, Usvsn Sai Prashanth, Edward Raff, Aviya Skowron, Lintang Sutawika, and Oskar Van Der Wal. Pythia: A suite for analyzing large language models across training and scaling. In *International Conference on Machine Learning*, 2023.

[3] Jianfei Chen, Lianmin Zheng, Zhewei Yao, Dequan Wang, Ion Stoica, Michael Mahoney, and Joseph Gonzalez. ActNN: Reducing training memory footprint via 2-bit activation compressed training. In *International Conference on Machine Learning*, 2021.

[4] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. Training deep nets with sublinear memory cost, 2016.

[5] Yiming Chen, Yuan Zhang, Yin Liu, Kun Yuan, and Zaiwen Wen. A memory efficient randomized subspace optimization method for training large language models, 2025.

[6] Sakshi Choudhary, Utkarsh Saxena, and Kaushik Roy. OASIS: Online activation subspace learning for memory-efficient training, 2026.

[7] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL*, 2019.

[8] Tuan-Kiet Doan, Trung-Hieu Tran, Enzo Tartaglione, Nikola Simidjievski, and Van-Tam Nguyen. INSTANT: Compressing gradients and activations for resource-efficient training, 2026.

[9] Gemma Team. Gemma 2: Improving open language models at a practical size, 2024.

[10] Gemma Team. Gemma 3 technical report, 2025.

[11] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, et al. The Llama 3 herd of models, 2024.

[12] Mojan Javaheripi, Sébastien Bubeck, et al. Phi-2: The surprising power of small language models. Microsoft Research Blog, 2023.

[13] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, et al. Mistral 7B, 2023.

[14] Marisa Kirisame, Steven Lyubomirsky, Altan Haan, Jennifer Brennan, Mike He, Jared Roesch, Tianqi Chen, and Zachary Tatlock. Dynamic tensor rematerialization. In *International Conference on Learning Representations*, 2021.

[15] Vijay Anand Korthikanti, Jared Casper, Sangkug Lym, Lawrence McAfee, Michael Andersch, Mohammad Shoeybi, and Bryan Catanzaro. Reducing activation recomputation in large transformer models. In *Proceedings of Machine Learning and Systems*, 2023.

[16] Yanyi Li, Yimu Zhang, and Cong Fang. PRAC: Principal-random subspace for LLM activation compression and memory-efficient training, 2026.

[17] Xiaoxuan Liu, Lianmin Zheng, Dequan Wang, Yukuo Cen, Weize Chen, Xu Han, Jianfei Chen, Zhiyuan Liu, Jie Tang, Joseph Gonzalez, Michael Mahoney, and Alvin Cheung. GACT: Activation compressed training for generic network architectures. In *International Conference on Machine Learning*, 2022.

[18] Anton Lozhkov, Raymond Li, et al. StarCoder 2 and The Stack v2: The next generation, 2024.

[19] Roy Miles, Pradyumna Reddy, Ismail Elezi, and Jiankang Deng. VeLoRA: Memory efficient training using rank-1 sub-token projections, 2024.

[20] Guilherme Penedo, Quentin Malartic, et al. The RefinedWeb dataset for Falcon LLM: Outperforming curated corpora with web data, and web data only, 2023.

[21] Qwen Team. Qwen2.5 technical report, 2025.

[22] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners, 2019.

[23] Sahar Rajabi, Nayeema Nonta, and Sirisha Rambhatla. SubTrack++: Gradient subspace tracking for scalable LLM training. In *Advances in Neural Information Processing Systems*, 2025.

[24] Yara Shamshoum, Nitzan Hodos, Yuval Sieradzki, and Assaf Schuster. CompAct: Compressed activation training for memory-efficient fine-tuning. In *Proceedings of NAACL*, 2025.

[25] Team OLMo, Pete Walsh, et al. 2 OLMo 2 furious, 2024.

[26] Haocheng Xi, Han Cai, Ligeng Zhu, Yao Lu, Kurt Keutzer, Jianfei Chen, and Song Han. COAT: Compressing optimizer states and activations for memory-efficient FP8 training, 2025.

[27] An Yang, Anfeng Li, Baosong Yang, et al. Qwen3 technical report, 2025.

[28] Susan Zhang, Stephen Roller, et al. OPT: Open pre-trained transformer language models, 2022.

[29] Jiawei Zhao, Zhenyu Zhang, Beidi Chen, Zhangyang Wang, Anima Anandkumar, and Yuandong Tian. GaLore: Memory-efficient LLM training by gradient low-rank projection. In *International Conference on Machine Learning*, 2024.

[30] Hanqing Zhu, Zhenyu Zhang, Wenyan Cong, Xi Liu, Sem Park, Vikas Chandra, Bo Long, David Z. Pan, Zhangyang Wang, and Jinwon Lee. APOLLO: SGD-like memory, AdamW-level performance, 2024.

A Supplementary Evidence Tables and Figures

A.1 Complete Matched-Cohort Numbers

Table 5: Complete numeric memory-quality-time tradeoff for the matched cohorts. Memory, PPL, and time are compared only within each cohort. Single RTX 5090 under the protocol of Section 5; seed 42, except the GPT-2 Large+GaLore rows (three-seed means, Table 12).

Cohort	Method	Peak GiB	PPL	Time s	Mem saved %
GPT-2 Medium	baseline	24.251	17.3574	136.0	0.00
GPT-2 Medium	checkpointing	**8.328**	**17.3569**	182.2	**65.66**
GPT-2 Medium	STAC-R full	22.019	17.6105	265.4	9.20
GPT-2 Medium	Turbo-fast	22.002	17.5319	**148.9**	9.27
Pythia-1.4B	baseline	12.721	10.8607	318.7	0.00
Pythia-1.4B	STAC-R full	**11.803**	**10.9675**	604.0	**7.22**
Pythia-1.4B	Turbo-fast	11.840	11.0316	**398.0**	6.93
GPT-2 Large + GaLore	baseline	14.614	15.0092	656.3	0.00
GPT-2 Large + GaLore	GaLore	10.894	**16.2299**	**941.5**	25.46
GPT-2 Large + GaLore	STAC-R + GaLore	**9.591**	16.6596	1875.9	**34.37**
GPT-2 Large	baseline	14.776	17.4565	82.7	0.00
GPT-2 Large	STAC-R MLP-only	13.040	**17.6579**	**198.1**	11.75
GPT-2 Large	STAC-R MLP+attn.	**12.546**	17.7873	371.3	**15.09**

A.2 Positioning Comparisons (Not Matched)

Table 6 places STAC-R beside other activation compressors across benchmarks. It is a positioning view, not a matched comparison: the INSTANT row uses the released INSTANT code path and the other rows use the local training harness, so quantitative memory claims in the main text use only within-protocol deltas.

Table 6: Cross-benchmark positioning grid; not a matched comparison. Each cell reports peak memory / quality; quality is validation accuracy (%) for SST-2 and validation perplexity for language modeling. On SST-2/BERT, the Turbo-fast and PRAC-untracked rows exceed the baseline peak.

Method	SST-2/BERT-base	GPT-2 Medium/WT-103	GPT-2 Large/WT-103	Pythia-1.4B/WT-103
Baseline	2.82 / 92.66	24.25 / 17.36	14.78 / 17.46	12.72 / 10.86
PRAC untracked r8	3.21 / 93.00	22.01 / 18.42	18.03 / 18.56	–
INSTANT ov15†	4.93 / 92.72	8.13 / 20.99	17.99 / 18.24	–
STAC-R full	**2.54 / 93.12**	22.02 / 17.61	**13.04 / 17.66**	**11.80 / 10.97**
STAC-R Turbo-fast	3.21 / 92.55	**22.00 / 17.53**	13.11 / 17.45	11.84 / 11.03

Peak numbers are allocator peaks from the audited source rows; quantitative memory claims in the text use within-protocol deltas. INSTANT† uses the released INSTANT code path; other rows use the local training harness. Dashes mark cells without a completed comparable run. Appendix Table 14 reports the SST-2 per-step SVD/TT/Tucker timing breakdown.

A.3 Prediction-Check and Mechanism Figures, Fidelity Telemetry, and Transfer Check

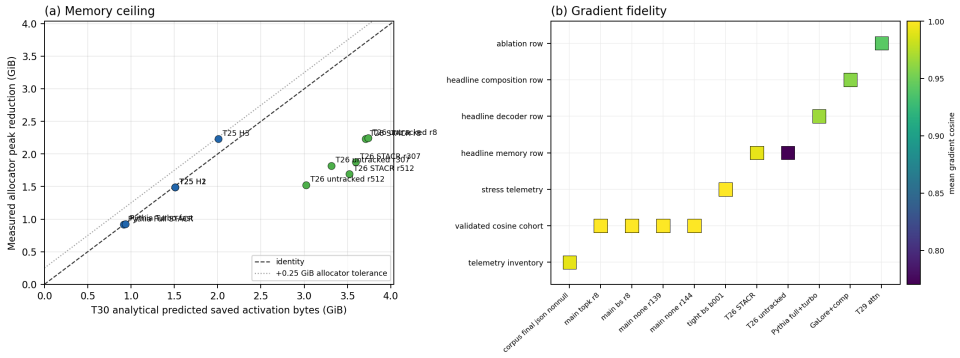


Figure 1: Quantitative target-surface check. Left: predicted saved bytes versus allocator/NVML realization. Right: gradient cosine from 240-step same-shape replays.

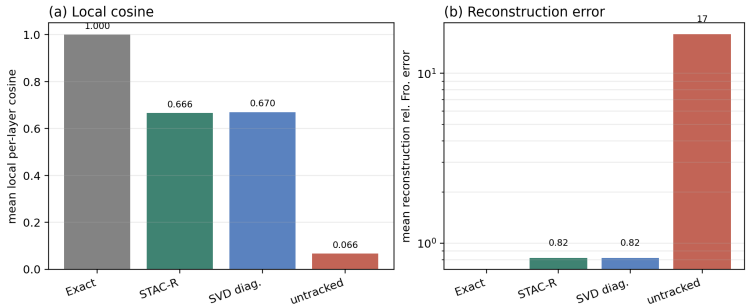


Figure 2: Matched-surface mechanism comparison on the GPT-2 Medium rank-8 MLP-linear surface. Exact activations, STAC-R, batch-local SVD, and an untracked subspace are measured under the same local protocol.

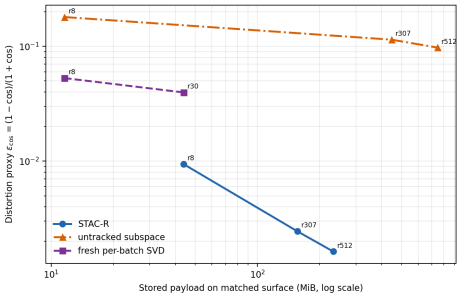


Figure 3: Empirical rate-distortion frontier on the GPT-2 Medium MLP-linear surface. Rate is stored payload; distortion is $\epsilon_{\cos} = (1 - \cos)/(1 + \cos)$.

Table 7: Measurement map for mechanism evidence.

Quantity	Value	Location	Protocol
STAC-R gradient cosine	0.9813	Tab. 8	240-step full-model replay
Untracked gradient cosine	0.6949; 0.6950 seed-43	Tab. 8	matched-rank replay
Iso-byte SVD cosine	0.9236	Fig. 5	matched stored bytes
Local mechanism cosine	STAC-R 0.666; SVD 0.670; untracked 0.066	Fig. 2	same-layer replay
Tracked-vs-untracked envelope	0.819 bound; 0.061 proxy	App. B.3	explained-variance envelope
Residual correction	$\gamma_\epsilon = 0.193$ r8; 0.099 r307	Tab. 10	residual on/off replay

Table 8: Short same-shape gradient-cosine replay.

Family	Row	Rank	Mean cosine	Worst cosine
GPT-2 Medium STAC-R	r8	8	0.9813	0.9439
GPT-2 Medium STAC-R	r307	307	0.9951	0.9853
GPT-2 Medium STAC-R	r512	512	**0.9967**	**0.9902**
GPT-2 Medium untracked subspace	r8	8	0.6949	0.6948
GPT-2 Medium untracked subspace	r307	307	0.7947	0.7944
GPT-2 Medium untracked subspace	r512	512	**0.8222**	**0.8219**
GPT-2 Medium untracked subspace	r8 seed 43	8	0.6950	0.6949
Pythia	STAC-R full	8	**0.9672**	**0.9015**
Pythia	Turbo-fast	8	0.9650	0.8951
Composition	STAC-R+GaLore	8	0.9588	0.8765
GPT-2 Large	MLP+attention	8	0.9415	0.8245

OpenWebText same-shape transfer check. At the GPT-2 Medium reported shape on OpenWebText (b=8, seq=1024, 500 steps, seeds 42/123/456), STAC-R r8 reduces allocator peak by 8.34% on every seed (26.585 GiB to 24.368 GiB), with mean final training-batch loss delta $+0.0296$ and $1.73\times$ time. This cross-corpus transfer row records final training-batch loss/PPL; validation-quality claims use the matched validation PPL cohorts.

Table 9: Comparator coverage by evaluation question.

Question	Comparator set	Location
Backward fidelity	exact, STAC-R, SVD, untracked; OASIS and PRAC bases	Fig. 2; Tabs. 8, 24, 25; Fig. 5
Factorization cost	SVD, TT, Tucker	Tab. 14
Residual channel	full, low-rank-only, sparse-only, residual on/off, residual budget	Tabs. 19, 10, 24, 25
Speed/composition	Turbo-fast, recomputation frontier, GaLore, SubTrack++, accumulation	Tabs. 2, 1, 12, 17, 16, 18, 20
Scale	18 models to 8B, sequence length, DDP, FSDP2, TP=2	Tabs. 3, 27, 21; Sec. 4.3
Operational guidance	wrapped surface $\times$ rank	Tab. 26
Related work	INSTANT, PRAC, OASIS, SVD/TT/Tucker, CompAct, ActNN, GACT, VeLoRA, COAT, RSO, APOLLO, DTR	Tabs. 6, 13, 15, 24
Operating region	checkpointing, Qwen/Llama residency, TP=2, GNN, small-fraction rows	Tabs. 4, 23, 21, 22; Sec. 7

A.4 Per-Batch Matrix-Factor Storage Ceiling

Remark 2 (Per-batch matrix-factor storage ceiling)**.** *For a batch-local rank-k factorization of an activation matrix $A \in \mathrm{R}^{B\times N}$ that stores two dense factors and singular values for the current batch, the stored scalar count is $k(B+N+1)$. The maximum compression ratio over $k \geq 1$ is therefore*

$$\rho_{\max}(B,N) = \frac{BN}{B+N+1}, \tag{7}$$

attained at $k = 1$. This ceiling explains why batch-local matrix-factor storage of this form is structurally limited in small-batch or short-sequence regimes: the factor payload includes batch- and feature-side factors every step. STAC-R *uses a persistent basis Q and stores only per-batch coefficients plus the sparse residual, so the basis cost is amortized rather than re-materialized for every activation.*

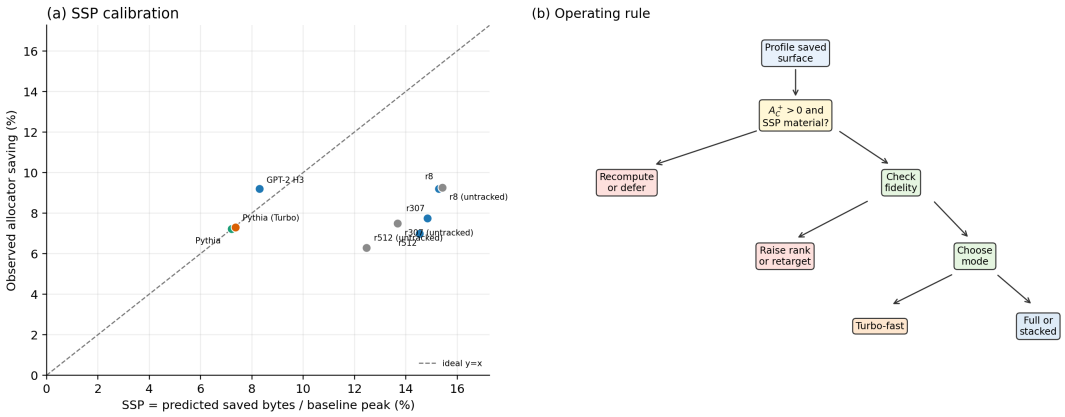


Figure 4: Saved-Surface Predictor decision map. The left panel calibrates SSP against observed allocator saving; the right panel shows the operating rule used in Table 4.

A.5 Phase Attribution Timings

The GPT-2 Medium STAC-R r8 240-step trace records 160.6 s wall time, with instrumented forward/backward/optimizer scopes totaling 88.3/52.6/4.0 s and a nested residual-correction subphase totaling 62.3 s across 11,520 layer calls. The Pythia-1.4B trace gives the same phase ordering at lower absolute time: 106.9 s wall time, 43.5/38.3/8.6 s forward/backward/optimizer scopes, and a 27.7 s nested residual-correction subphase.

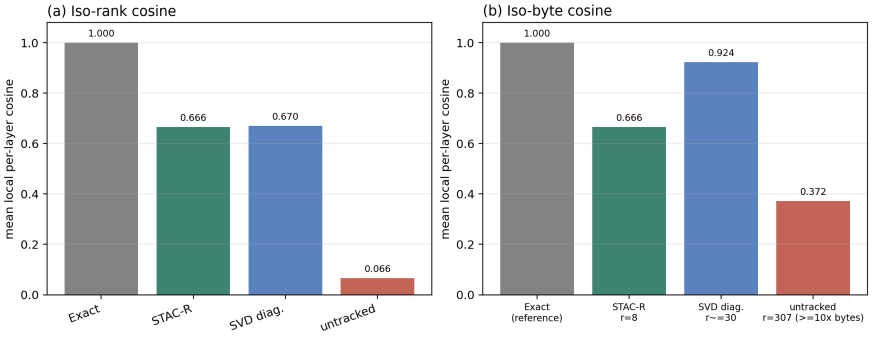


Figure 5: Iso-byte companion to Figure 2. SVD r=30 matches the stored payload of STAC-R r=8; the untracked r=307 cell is an over-budget upper-bound diagnostic.

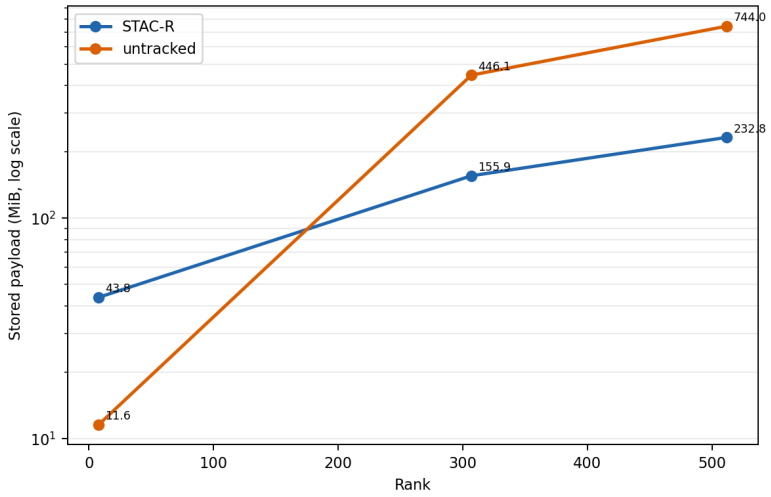


Figure 6: Stored payload growth versus rank on the GPT-2 Medium matched MLP-linear surface.

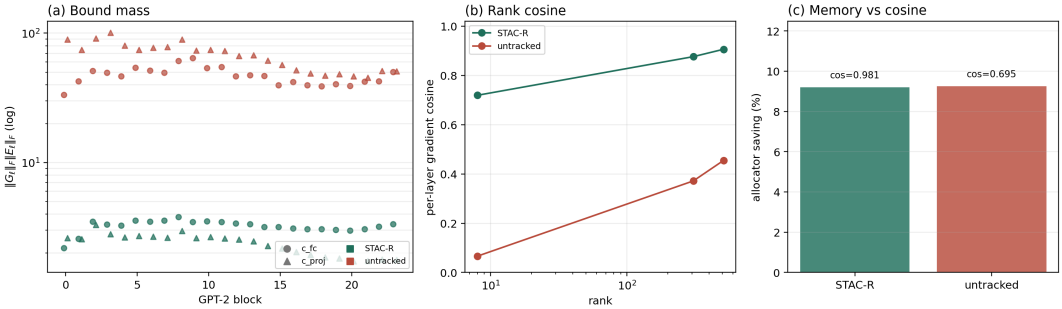


Figure 7: Rank-sweep mechanism on the GPT-2 Medium MLP-linear surface. Panels report Lemma-1 bound mass, local per-layer cosine, and rank-8 memory/cosine separation.

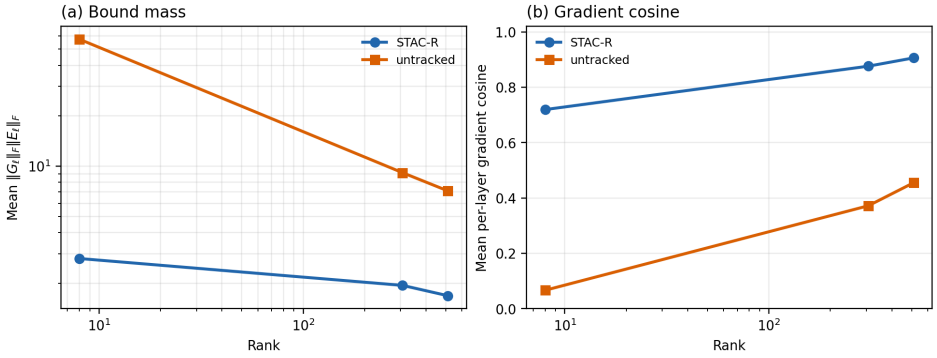


Figure 8: Per-layer allocation telemetry on the GPT-2 Medium 240-step same-shape replay.

INSTANT short-sequence saturation. In the L=128 official-code row used for Table 15, the released INSTANT setting saturates to 1.0× effective activation compression because the projector ranks reach the sequence dimension, while STAC-R r8 retains 88.7× activation compression on the same shape. This is the empirical counterpart of the per-batch factor-storage ceiling in Appendix A.4.

Distributed execution. The saved-tensor wrapper is local to a device replica. In addition to Table 21, a replicated `torch.nn.DataParallel` compatibility run completed on GPT-2 XL across 2× RTX

Table 10: Residual-on/off gamma-by-rank sweep on the GPT-2 Medium MLP-linear surface.

Rank	Cos. on	Cos. off	ϵ on	γ_ϵ
8	0.9813	0.9792	0.009435	**0.1930**
32	0.9869	0.9856	0.006588	0.1754
128	0.9922	0.9915	0.003927	0.1485
307	**0.9951**	**0.9948**	**0.002453**	0.0992

Table 11: Pythia-1.4B long-horizon quality/fidelity configurations. The 10k rows are the matched 5-seed quality cohort.

Row		PPL	PPL Δ	Mean cos.	Worst cos.	Time $\times$
5k STAC-R seed 42		11.4527	+2.56%	0.9901	0.9015	1.23
10k baseline 5-seed		13.172	0.00%			1.00
10k STAC-R 5-seed		**13.322**	**+1.14%**	**0.9917**	**0.8930**	1.24
10k Turbo-fast 5-seed		13.333	+1.22%	0.9912	0.8855	**1.22**
10k stddevs	base 0.106; STAC-R **0.047**					

5090 at batch 12 for 20 steps. These are compatibility measurements; Section 4.3 adds per-rank memory measurements under DDP and FSDP2. DDP correctness, tensor-parallel communication parity, and shared cross-rank basis protocols remain unmeasured.

Table 12: STAC-R and Turbo-fast with GaLore on GPT-2 Large, 5k steps and three seeds. Marginal rows are paired deltas over GaLore-alone.

Method	PPL	Peak GiB	Time s	Peak Δ%
baseline	15.0092 ± 0.0028	14.614	656.3	+0.00
GaLore	**16.2299 ± 0.0074**	10.894	**941.5**	-25.46
STAC-R + GaLore	16.6596 ± 0.0262	9.591	1875.9	-34.37
Turbo-fast + GaLore	16.5898 ± 0.0092	**9.587**	1257.9	**-34.40**
STAC-R marginal over GaLore	+0.4297 ± 0.0334	-1.302	+934.5	-11.96
Turbo-fast marginal over GaLore	**+0.3600 ± 0.0036**	**-1.307**	**+316.4**	**-12.00**

Table 13: Related-work taxonomy by compressed object and training problem.

Method	Compressed object	Standard training?	Distinction from STAC-R	Evidence
Untracked subspace	saved activations	yes	same wrapped surface; fixed basis without tracking or residual correction	same-surface local control
OASIS	saved activations + optimizer state	yes (reported)	reported whole-stack method; its online subspace basis is reimplemented here under the STAC-R backward, not its original pipeline	local basis reimplementation (Tab. 24)
INSTANT	per-batch activation factors	yes	batch-local factors; short-sequence saturation in our reproduction	Tab. 15
CompAct	saved activations	yes	activation compressor measured here in a small fp32 regime	bounded local measurement
ActNN	quantized saved activations	yes	quantization path; no tracked basis or factor-native residual contraction	published metrics
GACT	quantized saved activations	yes	controller/quantization path; reported through original-paper metrics	published metrics
GaLore	gradients / optimizer state	yes	optimizer-side projection; saved activations remain materialized	3-seed composition row
SubTrack++	gradients / optimizer state	yes	optimizer-side tracker; STAC-R adds activation savings after rank-4 SubTrack++	Tab. 18
APOLLO	optimizer-side state	yes	optimizer-side projection; composition axis rather than saved-activation method	literature positioning
RSO	optimization subproblem	no	changes the optimization problem through randomized subproblems	mechanism positioning
VeLoRA	token projections	yes	token-projection mechanism; no audited saved-tensor wrapper row	mechanism positioning
COAT	activations + optimizer state	yes	quantized whole-stack state; literature-positioned in this study	literature positioning
DTR	saved activations (evicted, rematerialized online)	yes (lossless)	online eviction and recomputation; stores no compressed state; not evaluated directly	literature positioning

Table 14: Same-pipeline per-batch factorization comparison on SST-2/BERT, batch 32, 3 epochs, no AMP and no checkpointing.

Method	Peak GB	Throughput tok/s	Accuracy	Act. comp.
Baseline	2.82	2628	92.66%	–
STAC-R	**2.54**	**730**	93.12%	24.7×
SVD (per-batch)	3.16	116	93.00%	438.6×
TT (per-batch)	2.61	117	**93.23%**	**2976**×
Tucker (per-batch)	3.19	76	92.78%	1352×

Table 15: Official-code INSTANT reproduction. Deltas are measured against each method's own baseline because the released INSTANT and STAC-R code paths use different harnesses; negative peak deltas indicate lower memory.

Benchmark	STAC-R	INSTANT ov15	INSTANT ov25	Seeds
SST-2/BERT	**+0.19 pp acc.; -11.2% peak**	-0.54 pp; +0.1% peak	-0.54 pp; +2.4% peak	3/3
GPT-2 Large/WT-103	**+1.4% PPL; -9.3% peak**	+21.8% PPL; +15.5% peak	+21.7% PPL; +7.8% peak	2/2
MRPC/BERT	**-0.010 F1; -13.4% peak**	-0.012 F1; +1.5% peak	-0.007 F1; +3.7% peak	4/3

Table 16: Matched-budget STAC-R residual versus pure-low-rank operating points. The STAC-R row is rank-8 plus sparse residual; pure LR uses the listed rank at matched activation-storage budget.

Model/task	Pure LR rank	Seeds	Hybrid tok/s	Pure LR tok/s	Gain
BERT-base/SST-2	139	20	**11927**	11109	**+7.4%**
BERT-large/SST-2	182	6	**6158**	4860	**+26.7%**
GPT-2 Large/WT-103	144	9	**12246**	8766	**+39.7%**
GPT-2 XL/WT-103	178	12	**4915**	3167	**+55.2%**

Table 17: Adaptive Turbo runtime measurements, 3 seeds each.

Config	Turbo quality	Control	Change	Speedup vs full
BERT-base SST-2	$92.96 \pm 0.18\%$	92.81% full	+0.15 pp	1.57×
BERT-large SST-2	$93.77 \pm 0.27\%$	93.92% baseline	-0.15 pp	1.22×
GPT-2 Large b=8	14.691 ± 0.006	14.586 full	+0.72% PPL	1.67×
GPT-2 Large b=11	14.546 ± 0.039	14.430 full	+0.80% PPL	1.63×
GPT-2 XL b=6	13.417 ± 0.012	13.315 full	+0.77% PPL	1.66×
GPT-2 XL seq=1024	12.442 ± 0.028	12.358 full	+0.68% PPL	1.57×

Table 18: SubTrack++ and STAC-R composition on SST-2/BERT-base, 3 seeds. SubTrack++ compresses optimizer/gradient state; STAC-R adds saved-activation compression.

Config	Accuracy	Peak alloc.	Peak reserved	vs rank-4
SubTrack++ baseline	$92.74 \pm 0.13\%$	2.711 GB	3.445 GB	–
SubTrack++ rank-4	$90.90 \pm 0.18\%$	2.207 GB	2.732 GB	–
SubTrack++ + STAC-R	$91.28 \pm 0.00\%$	**1.919 GB**	**2.406 GB**	**-13.1% / -11.9%**

Table 19: Sparse-only control. `residual_mode=none` is the low-rank-only row.

Mode	SST-2/BERT acc.	WT-103/GPT-2 Large PPL
Full STAC-R (rank 8 + sparse residual)	**92.89%**	**16.31**
Low-rank only (rank 8, no residual)	92.78%	16.33
Sparse-only top-k (0.5% budget)	92.70%	16.61

Table 20: Additional composition and GPT-2 XL scale measurements.

Measurement	Evidence	Use
Gradient accumulation	GPT-2 Large b=9 accum=2: 28.77 GB, PPL 13.806; STAC-R b=10 accum=2: **27.96 GB**, PPL 13.973.	effective batch 20 vs 18
GPT-2 XL OOM crossing	5k 3-seed: baseline+8bit b=5 PPL 13.19, 29.13 GB; STAC-R+8bit b=6 PPL 13.31, **28.64 GB**.	**capacity crossing**
GPT-2 XL 10k triangle	Seed-42 10k PPL: baseline b=5 13.278; Turbo b=6 13.308; full STAC-R b=6 **13.191**.	convergence check

Table 21: Tensor-parallel measurement on GPT-2 Large, seq=1024, TP=2 on 2× RTX 5090. Memory is per-rank peak.

Setting	Baseline	STAC-R	Delta
5k, b=2, ckpt+AMP	14.91 PPL / 5.61 GB	15.15 PPL / **4.87 GB**	**+1.6% PPL; -13.3% mem**
200 steps, b=2, ckpt+8bit	17.30 PPL / 5.61 GB	17.40 PPL / **4.87 GB**	**+0.6% PPL; -13.2% mem**
Max verified batch, ckpt+AMP	b=22	**b=26**	**+18.2% capacity**

Table 22: Audited regimes outside the memory-reducing operating region.

Regime	Evidence	Binding factor
Checkpoint-boundary compression	+11.2% PPL; +0.79 GiB peak	checkpoint boundary overhead
GNN target mismatch	0% memory saving; ≈-10% acc.	message-passing memory
Checkpointed Llama (32 GiB regime)	OOM or +8% to +39% peak	checkpointing removes target surface
Small activation-fraction fine-tuning (BERT/CompAct)	approximately unchanged peak on fp32 SST-2	small saved-activation fraction
GPT-2 XL seq=1024 Adaptive Turbo	b=3: +3.002 GiB, +2.21% PPL	batch crossing with quality variance
GPT-2 Large attention surface	+0.493 GiB saved vs MLP-only; +87.45% time	split-QKV overhead
GPT-2 checkpointing-compatible configuration	ckpt: 17.3569 PPL, 8.3277 GiB	recompute dominates memory
Qwen-2.5-1.5B (32 GiB shape)	5k: +66.23% peak; cos. 0.9909/0.9092	checkpointed baseline removes target surface
Llama-3.2-1B SwiGLU (32 GiB shape)	5k: +75.99% peak; cos. 0.9906/0.9061	checkpointed baseline removes target surface
Naive adaptive rank c_fc=4, c_proj=12	17.6566 PPL; 0.908 mean cosine	per-layer floor violation

A.6 Additional Matched Comparisons

This subsection collects the matched measurements referenced in Sections 4-7. Each table compares rows only with the baseline or reference of its own cohort; seed counts and RNG pinning are stated per table.

Table 23: Recomputation sweep on the SwiGLU models of Table 1: STAC-R at rank 8 on the MLP surface, bf16 with 8-bit AdamW, matched shapes (Llama $b = 1$, $s = 4096$; Qwen $b = 2$, $s = 1024$), seed 42, 24 steps. Top: peak change of STAC-R as only the baseline's recomputation setting is swept. Bottom: every arm relative to the uncheckpointed baseline.

Configuration	Llama-3.2-1B	Qwen-2.5-1.5B
*Baseline recomputation setting (*STAC-R *peak change)*		
No recomputation	-7.08%	-7.80%
Every second block recomputed	-4.34%	-4.78%
All blocks recomputed	$+0.03\%$	-0.16%
Relative to the uncheckpointed baseline		
Every second block recomputed	-18.48%	-18.14%
+ STAC-R on the other blocks	-22.01%	-22.09%
+ STAC-R on the recomputed blocks	-18.46%	-18.35%

Sparse recomputation schedules. In a separate 300-step GPT-2 Medium cohort ($b = 8$, $s = 1024$, three seeds), recomputing one block in 6, 8, 12, or 16 saves 11.30%, 8.47%, 5.65%, and 5.65% at $1.050\times$, $1.038\times$, $1.026\times$, and $1.027\times$ baseline time. Sparser schedules approach $1.00\times$ only as the recomputed set, and with it the saving, shrinks.

Basis controls and SSP across payloads. Table 24 holds the rank, wrapped surface, and factor-native backward fixed. The first two rows use the same Oja basis with and without the residual; the other rows substitute the reimplemented OASIS and PRAC bases and an untracked random basis. SSP differs across the five payloads by 0.10 percentage points and the measured reduction by 0.11 points; in this cohort, SSP is about 1.6 points below the measured reduction for every payload. At PRAC's recommended rank for linear layers ($\lfloor 0.3n \rfloor = 307$), PRAC stores 201.5 MiB of factors per step for $\epsilon_{\cos} = 0.0459$, while STAC-R at rank 128 stores 90.0 MiB for 0.0387 and at rank 307 reaches 0.0285 (GPT-2 Medium, $b = 2$, $s = 1024$, RNG pinned). In a rank-307 training arm (GPT-2 Medium, $b = 8$, $s = 1024$, 300 steps, three seeds; baseline 23.554 GiB, 73.5 s, PPL 15.5450), PRAC saves 7.94% at $1.04\times$ with $+0.61\%$ PPL and STAC-R saves 7.85% at $2.03\times$ with $+0.30\%$ PPL. Table 25 compares gradient error at matched stored bytes.

Table 24: Basis controls at matched rank and surface: GPT-2 Medium, $b = 8$, $s = 1024$, 300 steps, three seeds, rank 8 on the MLP surface; baseline 23.554 GiB, 73.7 s, PPL 15.5445. All rows use the same factor-native backward. OASIS (online Oja subspace with normalized step) and PRAC (principal-plus-random hybrid with interval refresh $T_1 = T_2 = 500$, per its Algorithm 1) are local reimplementations of the bases, not the original pipelines; at this horizon PRAC's refresh occurs only at initialization. Wrapped cosine is from a separate RNG-pinned probe ($b = 2$, 12 steps). SSP is computed from the byte ledger.

Basis	SSP (%)	Δ peak (%)	Time (×)	Δ PPL (%)	Wrapped cos.
STAC-R (Oja + residual)	7.82	-9.41	1.70	$+0.92$	0.8808
Low-rank only (Oja, no residual)	7.92	-9.51	1.06	$+1.66$	0.8369
OASIS (reimplemented)	7.92	-9.52	1.07	$+1.72$	0.8288
PRAC (reimplemented)	7.92	-9.51	1.00	$+2.02$	0.8143
Untracked random	7.92	-9.51	0.99	$+1.64$	0.5377

Table 25: Gradient error at matched stored bytes: GPT-2 Medium MLP surface, $b = 2$, $s = 1024$, RNG pinned; $\epsilon_{\cos} = (1 - \cos)/(1 + \cos)$, lower is better. Each control is interpolated to the exact byte budget of the STAC-R point. STAC-R is lowest at every budget; the low-rank-only column spends the residual's bytes on additional rank.

Stored (MiB)	STAC-R	Low-rank only	OASIS	PRAC	Untracked
8.6	0.0673	0.0823	0.0917	0.0942	0.2953
11.3	0.0629	0.0793	0.0907	0.0910	0.2916
27.0	0.0522	0.0683	0.0836	0.0796	0.2677
90.0	0.0387	0.0513	0.0689	0.0609	0.2208

Residual cost. In a separate three-seed residual-budget cohort (GPT-2 Medium, $b = 8$, $s = 1024$, 300 steps; baseline 23.554 GiB, 73.7 s, PPL 15.5484), disabling the residual changes the peak reduction from 9.41% to 9.51%, the PPL change from $+0.91\%$ to $+1.63\%$, and wall time from $1.69\times$ to $1.05\times$; a 1% residual budget gives 9.26%, $+0.71\%$, and $1.83\times$. In a separate three-seed timing comparison, disabling the residual reduces wall time from 121 s to 73.2 s, whereas refreshing the basis every step instead of every 100 steps changes it by 3.6% (124.9 versus 120.6 s). Replacing the chunked exact top-k selector with a single-chunk selector reduces full STAC-R from $1.681\times$ to $1.376\times$ baseline time at the same reported peak (-9.41%) and nearly unchanged PPL (15.689 versus 15.688; two seeds). In matched RNG-pinned replays on the modern architectures ($b = 1$, $s = 512$ for the SwiGLU models; $b = 2$, $s = 1024$ for GPT-2 Medium; 12 steps), toggling the sparse residual on at fixed rank and seed reduces wrapped-layer gradient error $\epsilon_{\cos}$ by 27.6% (rank 8, cosine $0.84329 \to 0.88406$), 26.3% (rank 32, $0.87167 \to 0.90384$), and 25.1% (rank 128, $0.90629 \to 0.92900$) on GPT-2 Medium; by 26.3% (rank 8, $0.58762 \to 0.67875$) and 21.4% (rank 128, $0.76050 \to 0.80679$) on Llama-3.2-1B; by 41.6% (rank 8, $0.61380 \to 0.75464$) and 36.4% (rank 128, $0.77031 \to 0.84762$) on Qwen-2.5-1.5B; and by 45.8% (rank 8, $0.66002 \to 0.80007$) and 40.6% (rank 128, $0.79153 \to 0.87072$) on Qwen3-1.7B.

Basis freeze. On a 240-step GPT-2 Medium b=8 seq=1024 replay, default tracking gives cosine 0.9860 / PPL 17.853 versus frozen-after-warmup cosine 1.0 / PPL 17.856 (Δcosine$= 0.014$, ΔPPL$= 0.003$, Δwall$= +4.1\%$); disabling basis freeze lowers PPL on one seed. Frozen-after-warmup is the deterministic protocol when a fixed post-warmup basis is required, and default-policy changes require a matched multi-seed fidelity study.

Operational guidance. Table 26 gives measured costs for choosing the wrapped surface and rank. The MLP surface provides most of the saving at the lowest time cost; moving from rank 8 to 128 gives up 0.6 points of memory saving and recovers about half of the PPL gap, and adding attention contributes a further 2.3 points for about $0.6\times$ extra wall time.

Table 26: Wrapped surface and rank: STAC-R on GPT-2 Medium, $b = 8$, $s = 1024$, 300 steps, three seeds; baseline 23.554 GiB, 73.8 s, PPL 15.5433. Wrap the MLP surface first; raise the rank when quality is binding and add attention when memory is binding.

Wrapped surface	Rank	Δ peak (%)	Time ($\times$)	Δ PPL (%)
Attention only	8	-2.28	1.62	$+0.29$
Attention only	32	-2.18	1.62	$+0.24$
Attention only	128	-1.79	1.63	$+0.16$
MLP	8	-9.41	1.69	$+0.94$
MLP	32	-9.29	1.70	$+0.72$
MLP	128	-8.79	1.76	$+0.48$
MLP + attention	8	-11.72	2.30	$+1.55$
MLP + attention	32	-11.50	2.32	$+1.15$
MLP + attention	128	-10.60	2.38	$+0.70$

Distributed arms. Under DDP (GPT-2 Medium, $b = 4$ per rank, three seeds, all ranks within 0.03 GiB), Turbo-fast alone reduces per-rank peak by 8.94%, 8.92%, 8.95%, and 8.94% at world sizes 1, 2, 4, and 8, and composed with 12 recomputed blocks by 32.30%, 32.33%, and 32.38% at world sizes 1, 2, and 4, against 28.55% for recomputation alone. Table 27 gives the FSDP2 arms: Turbo-fast adds to recomputing every fourth block (7 of 28) at both world sizes, and with every block recomputed it changes per-rank peak by at most 0.07 points.

Table 27: FSDP2 per-rank peak change (%) on Qwen-2.5-1.5B: 8×RTX 5090 host, $b = 2$ per rank, PyTorch AdamW with bf16 states, three seeds, every rank measured. Each column is relative to the uncheckpointed baseline at that world size. World sizes 1 and 2 reproduced byte-identically on a second machine. At world size 4, Turbo-fast alone gives -9.99% (no second-machine check).

Arm	World size 1	World size 2
Turbo-fast	-6.51	-8.34
7 of 28 blocks recomputed	-7.44	-9.75
7 of 28 blocks recomputed + Turbo-fast	-12.35	-15.95
All blocks recomputed	-20.74	-38.78
All blocks recomputed + Turbo-fast	-20.70	-38.71

LoRA and QLoRA surface crossover. Parameter-efficient adapters alter the resident saved-activation surface in the same way recomputation does: a LoRA-frozen linear layer requires no ∇W, so autograd does not save its full input activation and $\mathrm{SSP} \to 0$ when the remaining surface is small. Across three-seed RTX 5090 cohorts, full fine-tuning with STAC-R reduces BERT-base peak memory by 17.96% ($13.241 \to 10.863$ GiB at $b = 64$, $s = 512$), 17.65% ($7.059 \to 5.813$ GiB at $b = 32$, $s = 512$), and 15.20% ($2.460 \to 2.086$ GiB at $b = 32$, $s = 128$). Under LoRA, where the base linear inputs are no longer saved, adding STAC-R changes BERT-base peak by $+2.52\%$ ($9.736 \to 9.981$ GiB), $+1.25\%$ ($5.197 \to 5.262$ GiB), and -3.40% ($1.792 \to 1.731$ GiB) on those shapes, while still saving -1.63% ($20.916 \to 20.574$ GiB, versus -9.42% from $23.520 \to 21.304$ GiB under full fine-tuning) on the wider GPT-2 Medium shape ($b = 8$, $s = 1024$). Conversely, QLoRA (4-bit base weights) reduces BERT-base peak relative to LoRA by -11.4% at $b = 32$, $s = 128$ ($1.683 \to 1.491$ GiB) where parameters bind, but increases peak by $+15.4\%$ at $b = 32$, $s = 512$ ($4.865 \to 5.613$ GiB) and $+21.8\%$ at $b = 64$, $s = 512$ ($9.129 \to 11.119$ GiB) as the saved-activation surface dominates.

Batch capacity, sequence-length composition, and attention kernels. On a 32 GiB RTX 5090 with Qwen-2.5-1.5B ($s = 1024$, bf16 autocast, 8-bit AdamW), full STAC-R raises the maximum trainable batch size from $b = 5$ (28.517 GiB) to $b = 6$ (29.710 GiB, $+20\%$ batch capacity), while Turbo-fast raises the maximum batch from 4 to 6 and increases throughput from 14,409 to 17,374 tokens/s. In the H100 Qwen-2.5-7B sequence-length sweep ($b = 1$, bf16, two seeds), composing 6 recomputed blocks with Turbo-fast on the remaining blocks changes peak by -0.01% ($1.022\times$ time) at $s = 1024$ (baseline 42.774 GiB; all blocks recomputed: -0.07% at $1.124\times$), -8.40% at $0.991\times$ time at $s = 2048$ (baseline 46.685 GiB; matching all-recomputed -8.42% at $1.166\times$ with 17.7% higher throughput), and -16.49% at $0.981\times$ time at $s = 4096$ (baseline 64.888 GiB; all-recomputed -34.03% at $1.215\times$). Finally, on Qwen-2.5-1.5B ($b = 1$, $s = 4096$), Turbo-fast removes the same absolute MLP byte volume under both unfused eager attention (2.223 GiB removed from a 57.273 GiB baseline, -3.88%; -22.10% composed with 6 recomputed blocks) and fused SDPA (2.237 GiB removed from a 26.111 GiB baseline, -8.57%; -18.80% composed); all main-text cohorts use the fused SDPA kernel so percentage savings are never inflated by an unfused attention denominator.

Training-horizon quality scaling and under-update control. In a matched GPT-2 Medium WikiText-103 horizon sweep ($b = 8$, $s = 1024$, three seeds per horizon, 6 blocks recomputed plus Turbo-fast), the validation PPL delta grows with step count from $+1.16\%$ at 400 steps to $+1.42\%$ at 2,000 steps, $+1.95\%$ at 5,000 steps, and $+2.41\%$ at 10,000 steps, motivating the long-horizon validation recommendation in Section 7. Conversely, on 600-step modern-decoder runs where compressed arms exhibit negative PPL deltas (lower PPL than baseline), an explicit control that freezes the same MLP projections without compression reproduces and exceeds the PPL drop by $1.7\times$ to $3.6\times$; we therefore treat short-horizon decoder PPL drops as an under-update regularization artifact rather than a compression quality gain. Across six structurally distinct compressors and CPU activation offload on the same wrapped surface, the disjoint composition rule holds within 0.02 percentage points.

B Proofs and Derivations

Table 28: Derived consequences used in evaluation.

Issue	Consequence	Location
Multi-layer error	Root-sum-square block bound from $\|G_\ell\|_2 \|E_\ell\|_F$.	App. B.2; Fig. 7
Rank floor	Require $\kappa_G \|E_r\|_F / \|A\|_F \le \sqrt{1-\tau^2}$ for target cosine τ.	App. B.2; Tab. 22
Basis tracking	Tracked-vs-untracked bound/proxy: 0.819 / 0.061.	Fig. 2; Tab. 8
Residual correction	Positive γ_ϵ across ranks; effect shrinks with rank.	Tabs. 10, 29; App. B.3
Scheduler	Pre-optimizer cosine is scheduler-invariant; quality uses matched horizons.	Tabs. 1, 11
Optimizer stacking	Saved activations and optimizer state are separate byte surfaces.	Tab. 12; App. B.5
Layer allocation	Allocate by $\|G_\ell\|_F \|E_\ell\|_F$ subject to a cosine floor.	Fig. 8; App. B.6

This appendix supplies formal derivations for the main theorem/lemma and for the operational consequences summarized in Table 28: multi-layer aggregation, rank selection, matched-surface tracked-vs-untracked comparison, residual correction, scheduler-invariant gradient fidelity, activation/optimizer surface separability, and layer-kind compressibility. References to W, S_C, A_C, A_C^+, E, G, and $\widehat{\nabla W}$ retain the meaning fixed in the main text. The appendix distinguishes algebraic results, where the proof is direct, from mechanism-and-calibration results, where a tighter first-principles derivation would require additional analysis.

Basis and comparator conventions. The symbol Q always denotes the persistent STAC-R training basis: an orthonormal matrix updated online by Oja-style streaming subspace tracking on saved activations, then re-orthogonalized and optionally frozen after warmup. Exact PCA/SVD appears as offline telemetry. When this appendix uses $\mathrm{ExplVar}_r(A)$, the quantity measures how much activation energy the tracked basis captures on the audited surface. Vanilla SVD, SVD/TT/Tucker per-batch factorization, and "local per-layer" cosines are comparators or measurement protocols with their own runtime semantics; STAC-R training uses the amortized Oja-tracked basis.

B.1 Main Theorem and Lemma Proofs

Target-surface memory ceiling. Let $\mathcal{T}$ be the set of saved-activation tensors intercepted by the compressor in one training step. By definition,

$$W(M, \mathrm{cfg}) = \sum_{T \in \mathcal{T}} |T|$$

is the uncompressed byte total of that wrapped surface, where $|T|$ denotes byte size, and $S_C(M, \mathrm{cfg}) \ge 0$ is the byte total of the compressor payload replacing the same tensors. Persistent bases, temporary workspaces, allocator fragmentation, and other non-surface terms are excluded from this payload ledger and measured empirically. The activation-side byte change is therefore

$$A_C(M, \mathrm{cfg}) = W(M, \mathrm{cfg}) - S_C(M, \mathrm{cfg}).$$

Since $S_C(M, \mathrm{cfg}) \ge 0$, we have $A_C(M, \mathrm{cfg}) \le W(M, \mathrm{cfg})$. Taking the positive part gives

$$A_C^+(M, \mathrm{cfg}) = \max\{A_C(M, \mathrm{cfg}), 0\} \le W(M, \mathrm{cfg}).$$

Thus any positive byte saving attributable solely to the wrapped saved activations is upper bounded by the wrapped surface itself. Dividing by the baseline allocator peak $P > 0$ yields the target-surface saving-fraction ceiling $A_C^+/P \le W/P$. This is a byte-accounting ceiling; measured CUDA peak can be lower or higher because non-wrapped allocator terms are empirical.

Activation-error to gradient-error bridge. For a wrapped linear layer, let $A, \widehat{A} \in \mathrm{R}^{B \times N}$ and upstream gradient $G \in \mathrm{R}^{B \times d_{\mathrm{out}}}$. The exact and compressed weight-gradient contractions are

$$\nabla W = G^\top A, \qquad \widehat{\nabla W} = G^\top \widehat{A}.$$

With $E = A - \widehat{A}$,

$$\Delta \nabla W = \nabla W - \widehat{\nabla W} = G^\top (A - \widehat{A}) = G^\top E.$$

The spectral/Frobenius product bound and then Frobenius submultiplicativity give

$$\|\Delta \nabla W\|_F = \left\|G^\top E\right\|_F \le \|G^\top\|_2 \|E\|_F = \|G\|_2 \|E\|_F \le \|G\|_F \|E\|_F .$$

For the cosine lower bound, set $u = \nabla W$, $v = \widehat{\nabla W}$, and $\delta = u - v = \Delta\nabla W$. Assume $u \neq 0$ and define the relative gradient error

$$\epsilon_G = \frac{\|\delta\|}{\|u\|} < 1.$$

Then $v \neq 0$. Write $a = \langle v, \delta\rangle / \|u\|^2$ and $s = \|\delta\|^2/\|u\|^2 = \epsilon_G^2$. Since $u = v + \delta$,

$$\|u\|^2 = \|v + \delta\|^2 = \|v\|^2 + 2\langle v, \delta\rangle + \|\delta\|^2,$$

so $\|v\|^2/\|u\|^2 = 1 - 2a - s$. Also

$$\langle u, v\rangle = \langle v + \delta, v\rangle = \|v\|^2 + \langle \delta, v\rangle = (1 - a - s)\|u\|^2.$$

Therefore

$$\cos(u, v) = \frac{\langle u, v\rangle}{\|u\|\|v\|} = \frac{1 - a - s}{\sqrt{1 - 2a - s}}.$$

Squaring and rearranging gives

$$\cos(u, v)^2 = 1 - s + \frac{a^2}{1 - 2a - s} \geq 1 - s = 1 - \epsilon_G^2,$$

where the denominator is positive because $v \neq 0$. Hence

$$\cos(u, v) \geq \sqrt{1 - \epsilon_G^2}.$$

This is the sharp gradient-space statement. The triangle-inequality variant $(1 - \epsilon_G)/(1 + \epsilon_G)$ is looser and is used only where a conservative scalar bound is sufficient.

Activation-space relative-error bridge. The previous display is expressed in the relative gradient error ϵ_G. To connect it to activation reconstruction error, assume $A \neq 0$ and $G^\top A \neq 0$. Then

$$\epsilon_G = \frac{\left\|G^\top E\right\|_F}{\|G^\top A\|_F} \leq \frac{\|G\|_2 \, \|E\|_F}{\|G^\top A\|_F} = \kappa_G \frac{\|E\|_F}{\|A\|_F}, \qquad \kappa_G := \frac{\|G\|_2 \, \|A\|_F}{\|G^\top A\|_F}.$$

Thus the universal activation-space bridge includes a conditioning factor κ_G. The evaluation therefore reports measured gradient cosine directly and uses activation explained variance as telemetry rather than a universal fixed threshold.

Table 29: Empirical check of the gradient-space bound across 108 probed layers. The bound column is the mean value of $\sqrt{1 - s}$ from the corrected relative-error form; $p \to s$ reports the mean projected-only error proxy before and after sparse residual correction.

Metric	BERT-base	GPT-2 124M	GPT-2 Large
Layers checked	12/12	24/24	72/72
Mean $\cos_{\text{full}}$	0.869	0.996	0.985
Mean bound $\sqrt{1 - s}$	0.842	0.996	0.984
q-bridge MAE	3.45×10^{-5}	6.87×10^{-6}	5.00×10^{-6}
Mean $p \to s$	$0.855 \to 0.287$	$0.390 \to 0.008$	$0.640 \to 0.032$
Residual error drop	3.0×	50.6×	20.2×
Bound holds	12/12	24/24	72/72

B.2 Multi-Layer Aggregation and Rank Selection

Multi-layer aggregation. For each wrapped layer $\ell = 1, \ldots, L$, define

$$E_\ell = A_\ell - \widehat{A}_\ell, \qquad \Delta_\ell = G_\ell^\top E_\ell.$$

Vectorize each Δ_ℓ and concatenate the layer-local blocks:

$$\Delta_{\text{all}} = \left[\text{vec}(\Delta_1), \ldots, \text{vec}(\Delta_L)\right].$$

Because the blocks live in disjoint parameter-gradient coordinates,

$$\|\Delta_{\text{all}}\|_2^2 = \sum_{\ell=1}^{L} \|\Delta_\ell\|_F^2 .$$

Applying Lemma 1 layerwise gives

$$\|\Delta_{\text{all}}\|_2 = \left(\sum_{\ell=1}^{L} \|\Delta_\ell\|_F^2\right)^{1/2} \leq \left(\sum_{\ell=1}^{L} \|G_\ell\|_2^2 \, \|E_\ell\|_F^2\right)^{1/2}.$$

This immediately implies both

$$\|\Delta_{\text{all}}\|_2 \leq \sum_{\ell=1}^{L} \|G_\ell\|_2 \, \|E_\ell\|_F \leq L \max_\ell \|G_\ell\|_2 \, \|E_\ell\|_F$$

and the sharper blockwise bound

$$\|\Delta_{\text{all}}\|_2 \leq \sqrt{L} \max_\ell \|G_\ell\|_2 \, \|E_\ell\|_F .$$

The concatenated gradient-vector statement requires no cancellation or independence assumption. Summing error tensors in a shared vector space introduces cross terms; a $\sqrt{L}$ rate under that model requires additional orthogonality or zero-cross-correlation assumptions. The stated result uses the concatenated blockwise statement above.

Rank-selection sufficient condition. Fix a target gradient cosine $\tau \in (0, 1)$ and let $\widehat{A}_r$ be a rank-r reconstruction with residual $E_r = A - \widehat{A}_r$. The activation-space bridge gives

$$\epsilon_G \leq \kappa_G \frac{\|E_r\|_F}{\|A\|_F}.$$

The sharp gradient-space cosine result gives

$$\cos(\nabla W, \widehat{\nabla W}_r) \geq \sqrt{1 - \epsilon_G^2}.$$

Thus it is sufficient for $\cos \geq \tau$ that

$$\kappa_G \frac{\|E_r\|_F}{\|A\|_F} \leq \sqrt{1 - \tau^2}.$$

Equivalently, if a rank-r basis captures activation energy fraction ρ_r so that $\|E_r\|_F^2 \leq (1 - \rho_r) \|A\|_F^2$, then a sufficient energy condition is

$$\rho_r \geq 1 - \frac{1 - \tau^2}{\kappa_G^2}.$$

This conservative floor is one input to allocation: κ_G is layer- and step-dependent, and allocator measurement determines whether the rank fits the target-surface ceiling.

B.3 Matched-Surface and Residual Consequences

Tracked-vs-untracked envelope. Let $0 < r < N$, let $Q \in \mathrm{R}^{N \times r}$ be an orthonormal tracked basis, and let $U \in \mathrm{R}^{N \times r}$ be a uniformly random orthonormal basis. Define the low-rank reconstruction errors

$$E_Q = A - AQQ^\top, \qquad E_U = A - AUU^\top.$$

If the tracked basis explains fraction $\mathrm{ExplVar}_r(A)$ of the activation energy, then

$$\|E_Q\|_F^2 = \big(1 - \mathrm{ExplVar}_r(A)\big) \, \|A\|_F^2 .$$

For the random subspace,

$$\mathrm{E}_U\left[\|E_U\|_F^2\right] = \left(1 - \frac{r}{N}\right) \|A\|_F^2 .$$

Thus, at the same layer and upstream gradient G, Lemma 1 gives the ratio against the random-subspace RMS product-bound envelope

$$\frac{\|G\|_F \, \|E_Q\|_F}{\|G\|_F \sqrt{\mathrm{E}_U[\|E_U\|_F^2]}} = \sqrt{\frac{1 - \mathrm{ExplVar}_r(A)}{1 - r/N}}.$$

When $\mathrm{ExplVar}_r(A) > r/N$, the tracked low-rank basis has a smaller Lemma 1 product-bound envelope than the random subspace. The residual channel can only improve this envelope when it further lowers reconstruction error. The main-text value 0.819 is this conservative envelope on the GPT-2 Medium matched surface. The reported 0.061 is a measured directional proxy ratio using $1 - \cos$; it is reported as empirical slack relative to the conservative envelope.

Residual correction bound. Let E_{lr} be the low-rank-only reconstruction error and E_{res} the error after adding the sparse residual channel. If

$$\|E_{\mathrm{res}}\|_F^2 \le (1-\gamma)\,\|E_{\mathrm{lr}}\|_F^2\,, \qquad 0 \le \gamma \le 1,$$

then, writing $\Delta\nabla W_{\mathrm{res}} = G^\top E_{\mathrm{res}}$, Lemma 1 yields

$$\|\Delta\nabla W_{\mathrm{res}}\|_F \le \|G\|_F\,\|E_{\mathrm{res}}\|_F \le \sqrt{1-\gamma}\;\|G\|_F\,\|E_{\mathrm{lr}}\|_F\,.$$

Therefore the residual-corrected gradient-error envelope is no larger than $\sqrt{1-\gamma}$ times the low-rank-only envelope. The residual table logs cosine for every point rather than true $\|E\|_F$, so the reported γ_ϵ is the cosine-equivalent proxy

$$\epsilon_{\cos} = \frac{1-\cos}{1+\cos}, \qquad \gamma_\epsilon = 1 - \left(\frac{\epsilon_{\cos,\mathrm{on}}}{\epsilon_{\cos,\mathrm{off}}}\right)^2.$$

This table therefore reports the residual channel as mechanism evidence.

Top-k residual energy retention. Let $R = A - CQ^\top$ be the dense low-rank residual and let R_k keep a selected set of residual entries. If the selected entries retain energy fraction β_E, i.e.

$$\|R_k\|_F^2 \ge \beta_E\,\|R\|_F^2\,, \qquad 0 \le \beta_E \le 1,$$

then the residual after sparsification obeys the exact tail bound

$$\|R - R_k\|_F^2 = \|R\|_F^2 - \|R_k\|_F^2 \le (1-\beta_E)\,\|R\|_F^2\,.$$

For true top-k by magnitude, R_k is the best k-sparse approximation in Frobenius norm, so no other k-entry residual can have a smaller tail energy. The relevant variable is retained residual *energy*; a fixed 0.5% entry budget has different effect depending on the residual energy distribution. This is the formal reason the sparse residual is reported as a correction channel paired with the learned basis.

B.4 Scheduler-Invariant Gradient Fidelity

Statement. Let $A \in \mathrm{R}^{B\times N}$ be a saved activation, $\widehat{A} \in \mathrm{R}^{B\times N}$ a reconstruction (e.g. $\widehat{A} = CQ^\top + R_s$ for STAC-R), and $G \in \mathrm{R}^{B\times d_{\mathrm{out}}}$ the upstream gradient at a wrapped linear layer. Assume $G^\top A$ and $G^\top \widehat{A}$ are nonzero. Let $\eta_t > 0$ be a positive scalar learning rate at training step t. Then

$$\cos\big(\eta_t\, G^\top A,\ \eta_t\, G^\top \widehat{A}\big) \;=\; \cos\big(G^\top A,\ G^\top \widehat{A}\big). \tag{8}$$

In particular, the pre-optimizer gradient-cosine fidelity bound of Lemma 1 is invariant to any positive scalar learning rate or scalar LR schedule applied identically to the true and compressed gradient contractions. This statement covers the scalar learning-rate step before optimizer preconditioning, momentum, or weight decay.

Lemma B.1 (Positive scalar invariance of cosine)**.** *For any nonzero $u, v \in \mathrm{R}^d$ and any scalar $\alpha > 0$,*

$$\cos(\alpha u,\ \alpha v) \;=\; \cos(u, v).$$

Proof. By definition of the cosine of two nonzero vectors,

$$\cos(\alpha u, \alpha v) \;=\; \frac{\langle \alpha u,\ \alpha v\rangle}{\|\alpha u\|\,\|\alpha v\|} \;=\; \frac{\alpha^2\langle u, v\rangle}{(\alpha\|u\|)(\alpha\|v\|)} \;=\; \frac{\alpha^2}{\alpha^2}\cdot\frac{\langle u, v\rangle}{\|u\|\,\|v\|} \;=\; \cos(u, v),$$

where the cancellation $\alpha^2/\alpha^2 = 1$ uses $\alpha > 0$ and is exact (no limit, no first-order approximation). □

Proof of (8). Set $u = G^\top A$ and $v = G^\top \widehat{A}$, both nonzero and viewed as elements of $\mathrm{R}^{N\cdot d_{\mathrm{out}}}$ via vectorization. The factor-native gradient contraction is linear in the saved activation, so

$$\eta_t\, G^\top A = \eta_t \cdot u, \qquad \eta_t\, G^\top \widehat{A} = \eta_t \cdot v.$$

Apply Lemma B.1 with $\alpha = \eta_t > 0$:

$$\cos(\eta_t u,\ \eta_t v) \;=\; \cos(u, v).$$

Substituting back gives (8). □

Corollary B.1 (Lemma 1 bound is LR-invariant)**.** *Assume* $\nabla W \neq 0$ *and let* $\epsilon = \|\Delta\nabla W\|_F / \|\nabla W\|_F < 1$*. Then the conservative cosine lower bound from Lemma 1,*

$$\cos\big(\nabla W,\ \widehat{\nabla W}\big) \ \geq\ \frac{1-\epsilon}{1+\epsilon},$$

is unchanged when both arguments are rescaled by any positive scalar η_t*. Hence validation-perplexity gaps from different positions of a long warmup or decay schedule require matched-scheduler interpretation. This corollary concerns the learning-rate-scaled gradient before optimizer transformations.*

Proof. The numerator $\langle u, v\rangle$ and denominator $\|u\|\|v\|$ in the definition of cos both scale by η_t^2 under uniform rescaling, so the ratio is unchanged. The relative-error envelope ϵ is itself a ratio of norms of ∇W-shaped tensors and is therefore positive-scalar-invariant in the same way. Combining with the elementary algebra of $(1-\epsilon)/(1+\epsilon)$ yields the stated bound. □

Empirical check. On the Pythia-1.4B 5k single-seed replay, the linear-warmup LR sweeps over a $10\times$ range (1.45×10^{-6} at step 499 to 1.45×10^{-5} at step 4999), yet measured post-warmup gradient cosine remains high, with mean file-level cosine 0.9901 and worst-case minimum 0.9015. This is the empirical content of Corollary B.1: the LR ramp preserves the measured gradient-direction fidelity, and the 5k vs. 2k PPL gap is interpreted as a scheduler-horizon comparison issue.

B.5 Activation/Optimizer Surface Separability

Statement. Let C_A be a saved-activation compressor (e.g. STAC-R) that modifies only saved-activation tensors, and let C_B be an optimizer-state compressor (e.g. GaLore) that modifies only optimizer/gradient-state buffers. Define the wrapped activation surface W_A and the wrapped optimizer surface W_B as the byte volumes intercepted by C_A and C_B respectively. Let S_A and S_B denote the corresponding compressed payloads. Then, at the byte accounting level,

$$\Delta_{\text{stack}} \ =\ \Delta_A + \Delta_B - \Delta_{\text{overlap}}, \qquad \Delta_A := W_A - S_A, \quad \Delta_B := W_B - S_B, \tag{9}$$

where $\Delta_{\text{overlap}} = 0$ when the wrapped surfaces are disjoint in the autograd / optimizer state graphs. The Lemma 1 gradient-fidelity bound for C_A remains intact at a fixed forward/backward step under composition with C_B.

Lemma B.2 (Disjoint-surface byte-accounting decomposition)**.** *Let a byte-accounting ledger at a fixed training-step event decompose as*

$$B \ =\ W_A + W_B + R,$$

where W_A *is the wrapped activation surface,* W_B *is the wrapped optimizer surface, and* R *collects all remaining residual memory (parameters, gradients, workspace, fragmentation, transient buffers). If* C_A *modifies only the bytes inside* W_A *and* C_B *modifies only the bytes inside* W_B*, and* $W_A \cap W_B = \emptyset$ *at the tensor-graph level, then*

$$B_A = S_A + W_B + R, \quad B_B = W_A + S_B + R, \quad B_{AB} = S_A + S_B + R.$$

Proof. By assumption C_A leaves every byte outside W_A unchanged. The bytes inside W_A are replaced by the compressed payload S_A of size $|S_A|$. Hence the byte ledger under C_A alone is the sum of (i) the compressed activation payload, (ii) the unmodified optimizer surface W_B, and (iii) the residual R, giving $B_A = S_A + W_B + R$. The argument for B_B is symmetric: C_B replaces the optimizer surface W_B with S_B and leaves the saved activation surface untouched, so $B_B = W_A + S_B + R$. Under composition, C_A and C_B act on disjoint byte regions by hypothesis, so each method replaces its own target at the byte-accounting level. The regions in this ledger are S_A, S_B, and R, giving $B_{AB} = S_A + S_B + R$. □

Proof of (9). Subtract B_{AB} from B:

$$B - B_{AB} = (W_A + W_B + R) - (S_A + S_B + R) = (W_A - S_A) + (W_B - S_B) = \Delta_A + \Delta_B.$$

Since the byte savings from each method individually are $B - B_A = W_A - S_A = \Delta_A$ and $B - B_B = W_B - S_B = \Delta_B$, the stacked byte saving equals the sum of the individual byte savings, which establishes $\Delta_{\text{stack}} = \Delta_A + \Delta_B$ with $\Delta_{\text{overlap}} = 0$ under disjoint surfaces. The general form $\Delta_{\text{stack}} = \Delta_A + \Delta_B - \Delta_{\text{overlap}}$ absorbs any overlap byte volume that both methods would otherwise claim credit for. □

Corollary B.2 (Lemma 1 survives optimizer-side composition)**.** *Lemma 1's gradient-fidelity bound, $\|\Delta\nabla W\|_F \le \|G\|_F \|E\|_F$, involves only the upstream gradient G and the activation reconstruction error $E = A - \widehat{A}$. Since C_B modifies neither G at the saved-activation contraction step nor the saved A or $\widehat{A}$, the bound is unchanged for the same forward/backward step when C_A is composed with C_B. Optimizer-side projection has its own quality behavior (e.g. subspace-projection error of GaLore) that must be reported separately.*

Proof. The activation-error-to-gradient-error bridge is derived purely from the linear identity $G^\top A - G^\top \widehat{A} = G^\top E$ at the wrapped linear layer. By hypothesis, C_B acts on optimizer-state buffers outside the saved-activation autograd subgraph, leaving A, $\widehat{A}$, G, and E unchanged at the wrapped layer. Therefore the right-hand side of $\left\|G^\top E\right\|_F \le \|G\|_F \|E\|_F$ is identical for the same forward/backward step under the composed run. The optimizer-side method may modify the post-step weight trajectory; that effect is captured downstream by the standard optimizer-side analysis (e.g. subspace-projection error bounds for GaLore) and is orthogonal to this same-step activation-side bound. □

Remark B.1 (Allocator-level realization)**.** *The decomposition in Lemma B.2 is exact for the specified byte ledger. CUDA peak is a max-over-time functional. In practice, allocator fragmentation, caching of intermediate workspaces, and the timing of free/reallocate events can produce a signed non-additive term Δ_{frag} such that the measured allocator peak satisfies*

$$\Delta_{\text{stack}}^{\text{measured}} \;=\; \Delta_A + \Delta_B - \Delta_{\text{overlap}} - \Delta_{\text{frag}},$$

where the sign and magnitude of Δ_{frag} are empirical. The theorem is thus a byte-accounting identity, and allocator/NVML audits remain the empirical realization, exactly as in Theorem 1's allocator statement.

Empirical check. The 3-seed 5k STAC-R+GaLore cohort on GPT-2 Large reports a marginal STAC-R allocator-peak reduction of 1.302 GiB (11.96%) relative to the GaLore baseline, consistent with the standalone STAC-R surface prediction within the measured allocator tolerance. The Lemma 1 telemetry row for the composition has mean gradient cosine 0.9588, comparable to the STAC-R-only GPT-2 Medium cosine 0.9813 (Table 8), consistent with Corollary B.2: optimizer-side composition preserves the same-step activation-side gradient-bound setting.

B.6 Layer-Kind Compressibility from the GELU Bottleneck

Status. Unlike Sections B.4 and B.5, the next derivation is mechanism plus calibration. The objective is to explain *why* the measured per-layer product-bound mass differs systematically between `c_fc` and `c_proj` on GPT-2 MLPs and *why* an untracked random subspace and STAC-R show qualitatively different layer-kind ratios. The numerical $1.44\times$ figure is an empirical calibration of the mechanism.

Statement. Consider a GPT-2 MLP block with hidden dimension H and expansion factor 4: `c_fc` consumes the H-dimensional pre-expansion stream, and `c_proj` consumes the $4H$-dimensional post-GELU stream. Let $s(\cdot)$ denote stable rank, defined as $s(X) = \|X\|_F^2 / \|X\|_{\text{op}}^2$.

Under the standard saturation behavior of GELU on roughly Gaussian inputs, GELU acts as a soft gate: components with small magnitude are attenuated disproportionately, while large-magnitude components pass through nearly linearly. Let $\alpha \in (0, 1)$ denote the effective fraction of pre-expansion variance suppressed at the post-expansion stage. The mechanism we test is

$$s(\texttt{c_proj input}) \;\le_{\text{eff}}\; (1-\alpha)\cdot s(\texttt{c_fc input}), \tag{10}$$

meaning that the post-GELU side can have lower effective stable rank despite the $4\times$ dimension increase. We use this as a calibrated mechanism hypothesis with the following consequences:

- For an untracked random subspace, the expected reconstruction Frobenius error under an isotropic random r-dimensional subspace scales as $\sqrt{1 - r/N}$, which depends on ambient dimension N and total activation energy. The within-method `c_proj`/`c_fc` ratio is therefore approximately constant in r, because both layers see the same random-subspace envelope shifted by dimension and measured energy factors. Empirically, this constant is $\approx 1.44\times$.

- For a learned subspace (STAC-R via Oja tracking), the basis aligns with the dominant directions of activation variance. At small rank, the higher pre-expansion stable rank of `c_fc` means per-rank-unit captured variance is *lower* on `c_fc` than on `c_proj`, but `c_fc` still carries disproportionate measured gradient mass (because $\|G_{c_fc}\|_F$ remains large in the telemetry). As r grows past the effective stable rank of `c_proj`, the post-GELU concentrated variance is captured and the measured product mass shifts back toward `c_proj`.

Mechanism derivation. We now justify each step.

Step 1 (GELU is a soft gate). The GELU activation $\mathrm{GELU}(x) = x \cdot \Phi(x)$ satisfies $\mathrm{GELU}(x) \approx 0$ for $x \ll 0$ and $\mathrm{GELU}(x) \approx x$ for $x \gg 0$, with smooth interpolation near zero. For input components $x \sim \mathcal{N}(0, \sigma^2)$, the squared output expectation is

$$\mathrm{E}\big[\mathrm{GELU}(x)^2\big] \;=\; \mathrm{E}\big[x^2\Phi(x)^2\big] \;<\; \mathrm{E}\big[x^2\big] \;=\; \sigma^2,$$

because $\Phi(x)^2 \le 1$ with equality only as $x \to \infty$. The suppression is most pronounced near $x = 0$, where $\Phi(0) = 1/2$. Thus the post-GELU per-component variance is strictly smaller than the pre-expansion per-component variance, with a multiplicative factor that we collect into $(1 - \alpha)$.

Step 2 (Coordinate-wise attenuation can reduce effective stable rank). Coordinate-wise attenuation by a factor $\beta_i \in (0, 1]$ on a matrix X rescales the ith coordinate's contribution to both $\|X\|_F^2$ and $\|X\|_{\mathrm{op}}^2$. If attenuation mostly suppresses diffuse small-magnitude directions while leaving dominant directions comparatively intact, then the Frobenius norm falls more than the operator norm and stable rank decreases. GELU is a plausible mechanism for that condition, and the effect is distribution-dependent; the analysis is calibrated by measured per-layer telemetry. Equation (10) is therefore the empirical mechanism being calibrated.

Step 3 (The untracked control sees isotropic subspace dimension). For an orthogonal projector $PP^\top$ onto a uniformly random r-dimensional subspace of R^N, the expected residual energy on a fixed activation matrix $A \in \mathrm{R}^{B\times N}$ is

$$\mathrm{E}\Big[\big\|A - APP^\top\big\|_F^2\Big] \;=\; \Big(1 - \frac{r}{N}\Big)\,\|A\|_F^2\,,$$

where the expectation is over the random subspace. This depends on the ambient dimension N and total activation energy; the spectral profile of A enters through the learned-subspace case below. The within-method `c_proj`/`c_fc` ratio under the random-subspace control is therefore approximately a constant function of r for $r \ll H$:

$$\frac{\mathrm{E}[\|E_{c_proj,\mathrm{rand}}\|_F]}{\mathrm{E}[\|E_{c_fc,\mathrm{rand}}\|_F]} \;\approx\; \sqrt{\frac{(1 - r/4H)\;\|A_{c_proj}\|_F^2}{(1 - r/H)\;\|A_{c_fc}\|_F^2}}.$$

For $r \ll H$, the dimension factor is close to constant; the remaining ratio is determined by measured activation-energy and upstream-gradient terms. In the GPT-2 telemetry those terms make the random-subspace product-bound ratio rank-flat near $1.44\times$; this is an empirical calibration of the measured GELU-layer telemetry.

Step 4 (Learned subspace tracks the spectrum). For an Oja-tracked orthonormal basis Q with explained variance $\mathrm{ExplVar}_r(A)$, the residual energy is bounded by $\|E_{\mathrm{Oja}}\|_F^2 \le (1 - \mathrm{ExplVar}_r(A))\,\|A\|_F^2$. Whenever stable rank is lower (post-GELU side), $\mathrm{ExplVar}_r$ can rise faster with r. The measured telemetry then determines which layer has larger $\|G_\ell\|_F\,\|E_\ell\|_F$: at small r, `c_fc` is heavier for STAC-R; at larger r, the post-expansion concentrated directions are covered and the product mass crosses over to a `c_proj`-heavier regime. This yields the rank-dependent and sign-reversing ratio observed for STAC-R; stable rank is one measured contributor to the crossover point.

Empirical calibration. Table 30 summarizes the measured per-layer product-bound ratios on the GPT-2 Medium MLP-linear surface.

Table 30: Per-layer product-bound ratio between `c_proj` and `c_fc` on the GPT-2 Medium MLP-linear surface. The random-subspace ratio is rank-invariant; STAC-R's ratio is rank-dependent and sign-reversing, consistent with the mechanism in Section B.6. The $1.44\times$ value is empirically measured.

Method	`c_proj`/`c_fc` product-bound ratio		
	$r = 8$	$r = 307$	$r = 512$
Random subspace	$1.44\times$	$1.44\times$	$1.44\times$
STAC-R (learned subspace)	$0.73\times$	$0.94\times$	$1.15\times$

Remark B.2 (Scope of the layer-kind result)**.** *The mechanism above explains the qualitative pattern if the measured post-GELU spectra are indeed more concentrated: the untracked control sees mostly ambient dimension and yields a rank-flat ratio, while a learned subspace tracks the spectrum and yields a rank-dependent ratio that crosses over as r exceeds the effective post-GELU stable rank. The exact $1.44\times$ calibration is an empirically measured constant; predicting it from first principles would require a stability analysis of GELU on the specific input distribution at each layer, including LayerNorm preconditioning. We report the layer-kind corollary as mechanism+calibration evidence tied to the measured telemetry.*

Empirical check. The per-method, per-rank, per-layer-kind product-bound telemetry on the GPT-2 Medium rank sweep gives the values shown in Table 30; they are the same numbers that drive the per-layer figure in the main text (left panel of Fig. 7). Together with the matched-surface untracked-subspace contrast (Table 28) and the per-layer allocation objective $\sum_\ell \|G_\ell\|_F \|E_\ell\|_F$ from Lemma 1, this gives a mechanistic account for the fixed-recipe adaptive schedule with `c_fc=4, c_proj=12`: it allocates against the within-method ratio while the Lemma 1 allocation rule first requires a per-layer cosine floor.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes].

   Justification: The abstract and Introduction state the target-surface memory ceiling, gradient-fidelity bridge, implemented STAC-R mechanism, positive cohorts, the Turbo-fast and partial-recomputation results, and the configurations in which recomputation leaves no saved surface to compress. The claim scope excludes universal memory savings, quality claims at 7B/8B scale (those cohorts report memory and runtime only), strong GSM8K/instruction-following evidence, and checkpointing dominance claims.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes].

   Justification: Section 7 is dedicated to audited operating-scope regimes, including the Qwen/Llama rows measured against checkpointed baselines, configurations whose saved surface is too small, checkpointing-compatible GPT-2, graph-message memory, small activation-fraction fine-tuning, the first adaptive-rank schedule, and the quality and distributed-measurement scope of the added cohorts.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [Yes].

   Justification: Section 3 states the target-surface theorem, gradient-error bridge, and corollaries with their scope. Appendix B gives the supporting derivations and marks mechanism-calibration claims separately from formal guarantees.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes].

   Justification: Section 5 gives hardware, precision, optimizer, memory-source, seed, and telemetry conventions, while the public repository records the complete configuration, seeds, and environment of each matched cohort (`configs/protocols.json`) and the per-run measurements, including platform, peak memory, and time (`paper_data/results`).

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [Yes].

   Justification: The public code repository linked on the first page contains the code, cohort configurations, run scripts, per-run measurement records, and the CSVs behind the paper tables needed to reproduce or audit the displayed configurations. The release is code/artifacts for the memory-compression method; upstream datasets and pretrained model weights stay with their original providers.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes].

Justification: The main text specifies the evaluated model families, tasks, sequence/batch shapes for boundary configurations, AMP/optimizer setup, allocator/NVML memory sources, and seed handling. The exact configuration-level settings are recorded in `configs/protocols.json` of the public repository.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [Yes].

   Justification: The main claims distinguish single-seed appendix measurements from multi-seed matched cohorts. The Pythia 10k quality/fidelity cohort is reported over five seeds; SST-2 and the 5k GaLore composition cohort are reported over three seeds; the added recomputation-frontier and RTX 5090 Turbo-fast cohorts use three seeds and the H100 cohorts two seeds; and remaining single-seed operating-scope measurements are labeled with explicit seed scope.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes].

   Justification: Section 5 states the hardware (a single RTX 5090 32 GiB host for the original cohorts; H100 GPUs and an $8\times$RTX 5090 host for the added cohorts) and measurement conventions. The per-run records in the public repository list the platform and wall time of each measurement.

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes].

   Justification: The work is a systems/theory study of training-memory compression on established public model and dataset assets. The release contains a method implementation and audit artifacts, with no deployed decision system or human-subject dataset.

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [Yes].

    Justification: The paper discusses efficiency benefits and operating-scope regimes where the evaluated recipe should be deferred. The contribution is a training-memory technique. The main societal risks are indirect: lower training cost can broaden access and can also reduce the cost of training harmful models.

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A].

    Justification: This work releases code and experimental artifacts for a memory-compression method. It relies on established upstream model/dataset assets and adds no pretrained model, image generator, or scraped dataset release.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [Yes].

Justification: The paper cites the creators of the model/dataset assets used for evaluation. The code loads each model and dataset from its public release, whose model or dataset card states its license and terms, and redistributes no weights or data; bundled third-party comparator code keeps its original license files (INSTANT, MIT; SubTrack++ driver, Apache-2.0).

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [Yes].

    Justification: The new assets are the STAC-R code, cohort configurations, run scripts, and measurement data in the public code repository. They are documented through its README, `configs/protocols.json`, and `paper_data`.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A].

    Justification: The work uses public model/dataset assets and contains no crowdsourcing or human-subject study.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A].

    Justification: The work uses public model/dataset assets and contains no human-subject research.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: [N/A].

    Justification: Pretrained language models are evaluation substrates for memory-compression experiments. The proposed method and reported results do not rely on an LLM-based methodological component.